Prague University of Economics and Business

Faculty of Informatics and Statistics

Comparison of Selected Efficient Deep Learning Techniques for Convolutional Neural Networks in Classification

BACHELOR THESIS

Study programme: Data Analytics

Author: Xuan Vi Phamová

Bachelor Supervisor: doc. Ing. Ondřej Zamazal, Ph.D.

Prague, May 2025

Acknowledgement

I would like to express my sincere gratitude to my thesis supervisor, doc. Ing. Ondřej Zamazal, Ph.D., for his invaluable guidance, expertise, and patience throughout the course of this work. His insightful feedback and professional support greatly contributed to the development and successful completion of this thesis.

I would also like to thank my family and friends for their continuous encouragement and understanding during my studies. Their support has been essential and deeply appreciated.

Abstract

The deployment of convolutional neural networks (CNNs) on resource-constrained edge devices requires the application of model compression techniques to balance performance, size, and inference time. This thesis investigates three key techniques — weight pruning, quantization, and knowledge distillation — evaluating their impact on CNN performance across various datasets and model architectures. Experimental results show that weight pruning, when applied to simpler custom CNNs, significantly reduces model size while slightly improving accuracy. However, excessive pruning in more complex pretrained models led to a considerable drop in performance, highlighting the need for more advanced strategies in complex architectures. Quantization techniques, particularly Post-Training Quantization (PTQ), proved highly effective in reducing model size by 75-90% and accelerating inference time by up to 1.5-8×, without substantial accuracy degradation. In contrast, Quantization-Aware Training (QAT) offered similar benefits but introduced minor accuracy losses and the risk of overfitting, particularly when applied to larger models like InceptionV3. Knowledge distillation demonstrated strong potential for transferring knowledge from larger teacher models to smaller student models, resulting in improved classification accuracy without increasing model size or inference time. These techniques offer promising solutions for optimizing CNNs, ensuring efficient performance while maintaining competitive accuracy.

Keywords

deep learning, convolutional neural networks, model compression, pruning, quantization, knowledge distillation, efficient deep learning techniques, image classification, pretrained models

Abstrakt

Nasazení konvolučních neuronových sítí (CNN) na zařízení s omezenými výpočetními prostředky vyžaduje použití technik komprese modelů k vyvážení výkonu, velikosti a rychlosti inferenčního zpracování. Tato bakalářská práce zkoumá tři klíčové techniky — prořezávání, kvantizaci a destilaci znalostí — s cílem vyhodnotit jejich dopad na výkon CNN napříč různými datovými sadami a architekturami modelů. Výsledky experimentů ukazují, že prořezávání vah výrazně snižuje velikost modelu a mírně zlepšuje správnost, pokud je aplikováno na jednodušší vlastní CNN modely. Naopak nadměrné prořezávání u složitějších předtrénovaných modelů vede k výraznému poklesu výkonu, což poukazuje na nutnost pokročilejších strategií u komplexních architektur. Kvantizační techniky, zejména kvantizace po trénování (Post-Training Quantization, PTQ), se ukázaly jako vysoce účinné. Snížily velikosti modelu o více než 75-90 % a zrychlily inferenční dobu až o 1,5-8×, aniž by došlo k významnému poklesu správnosti. Naproti tomu kvantizace s ohledem na trénink (Quantization-Aware Training, QAT) nabídla podobné výhody, ale vedla k drobným ztrátám správnosti a riziku přetrénování, zejména při aplikaci na větší modely jako je InceptionV3. Destilace znalostí prokázala silný potenciál pro přenos znalostí z větších modelů učitelů do menších studentských modelů, což vedlo ke zlepšení správnosti klasifikace bez zvýšení velikosti modelu nebo inferenčního času. Tyto techniky nabízejí slibná řešení pro optimalizaci CNN s cílem zajistit efektivní výkon při zachování konkurenceschopné správnosti.

Klíčová slova

hluboké učení, konvoluční neuronové sítě, komprese modelů, prořezávání, kvantizace, destilace znalostí, efektivní techniky hlubokého učení, klasifikace obrazů, předtrénované modely

Contents

A List of Figures

A List of Tables

A List of Programme Code Excerpts

List of Abbreviations

ANN	Artificial Neural Networks
CNN	Convolutional Neural Networks
QAT	Quantization-Aware Training
PTQ	Post-Training Quantization
KD	Knowledge Distillation
TF	Tensorflow
FP32	32-bit floating-point
FP16	16-bit floating-point
INT8	8-bit integers
FPS	Frames per second
NAS	Neural Architecture Search

Introduction

Deep learning has become one of the most powerful and fast-growing areas in modern computer science, especially in the field of artificial intelligence. Among its many applications, image classification has received a lot of attention and has seen significant progress. One of the key technologies behind this success is the Convolutional Neural Network (CNN), a type of deep learning model that is very effective in analyzing and understanding image data.

CNNs are designed to automatically learn features from images, therefore CNNs have been used in many areas, such as recognizing objects in photos, detecting diseases in medical scans and more. These networks have often reached or even surpassed human-level performance on many benchmark datasets.

However, there is one major challenge with CNNs. They often require a lot of computing power, memory and energy. As CNNs grow deeper and more complex to improve accuracy, they also become slower and harder to use in real-time applications or on devices with limited resources, such as smartphones, tablets, or embedded systems. This was the prompt to search for ways to make CNNs more efficient, while still keeping their high performance.

In response to this problem, many efficient deep learning techniques have been developed. These include methods that redesign the network structure to reduce the number of calculations, compress the models to use less memory, and improve the training process so that smaller models can learn just as well as larger ones. Some popular examples are depth wise separable convolutions, which are used in models like MobileNet[1] to reduce computation, and knowledge distillation, which trains a smaller model using the output of a larger model as guidance. Other approaches involve pruning unimportant parts of the network or reducing the precision of weights.

The thesis investigates the impact of three key efficient techniques—weight pruning, quantization, and knowledge distillation—on CNNs trained for image classification tasks. These techniques are compared in terms of their effects on model accuracy, inference time, and model size. The main results indicate that weight pruning can significantly reduce model size, though its impact on accuracy varies depending on the complexity of the model. Quantization, especially Post-Training Quantization (PTQ), leads to substantial reductions in model size and improvements in inference speed with minimal accuracy loss. In contrast, Quantization-Aware Training (QAT) offers similar benefits but can lead to minor accuracy losses and risks of overfitting. Knowledge distillation shows strong potential in transferring knowledge from larger models to smaller ones, improving classification accuracy without increasing model size or inference time.

[1] https://keras.io/api/applications/mobilenet/

The main goal of this thesis is to review, apply, and compare several selected of these efficient deep learning techniques on CNNs using classification tasks. The methodology involves a review of scientific literature to assess the current state and identify promising methods for further comparison. It also involves designing and running experiments, carefully observing how each method performs, and measuring various aspects of the models, such as accuracy, training time, inference time or number of parameters. Special attention is given to understanding the image datasets themselves—what kind of images they contain and how difficult they are to classify.

By comparing the results and discussing the findings, this work aims to provide a clear overview of which efficient techniques are most useful in different situations. This can help other researchers, engineers, and developers choose the right method when building deep learning models that need to work under time, space, or power constraints.

In the end, this thesis not only helps answer technical questions about the performance of efficient deep learning methods, but also contributes to the broader goal of making deep learning more accessible and practical for real-world applications—especially in places where high performance must be balanced with limited resources.

1 Convolutional Neural Networks

Artificial neural networks (ANNs) form the foundational framework for modern deep learning systems. Inspired by the structure and functioning of the human brain, ANNs are composed of interconnected nodes, or neurons, arranged in layers. Each neuron receives input, applies a transformation and passes the output to the subsequent layer. Over time, these networks have evolved to solve increasingly complex tasks across domains such as image recognition, natural language processing and reinforcement learning (Goodfellow et al., 2016).

The popularity of neural networks in the last decade can be attributed the availability of large datasets, the advancement in computational resources (especially GPUs), and improved training algorithms such as backpropagation combined with stochastic gradient descent. Deep neural networks, which consist of multiple hidden layers, are capable of learning hierarchical representations of data, enabling them to generalize well on complex tasks (Goodfellow et al., 2016).

ANNs typically fall into various categories based on architecture and application - feedforward networks for general regression/classification, recurrent networks for sequential data, and convolutional networks for image and spatial data. Among these, Convolutional neural networks (CNNs) have become the leading approach for visual recognition tasks (Goodfellow et al., 2016).

1.1 CNN Architecture

CNNs (LeCun et al., 1989) have fundamentally transformed the field of computer vision (Shapiro & Stockman, 2001). Their architecture, inspired by the biological processes of the visual cortex, is specifically designed to automatically and adaptively learn spatial hierarchies of features from input images. CNNs have become the standard for tasks involving image classification, object detection, and segmentation due to their ability to capture spatial using relevant filters. They are specifically designed to process data with a grid-like structure, such as images. They are called 'convolutional' because they use a mathematical operation called convolution, which is a specialized form of linear transformation (Goodfellow et al., 2016).

1.1.1 Convolutional Layer

At the core of CNNs is the convolutional layer, which applies a series of learnable filters across the spatial dimensions of the input image. Each filter is small spatially (e.g., 3x3 or 5x5) but extends through the full depth of the input volume. As the filter slides across the image, it performs an element-wise multiplication followed by a summation, producing a 2D activation map or feature map. The convolutional layers are responsible for detecting local patterns in the input, such as edges, corners, and textures. The operation involves

convolving a set of learnable filters (kernels) across the input tensor, producing feature maps that highlight the presence of these learned patterns (Goodfellow et al., 2016).

1.1.2 Activation Functions

Non-linear activation functions are essential to enable CNNs to approximate complex functions. The most widely used activation function is the Rectified Linear Unit (ReLU[2]) (Nair & Hinton, 2010) due to its computational efficiency and ability to mitigate the vanishing gradient problem. Variants such as Leaky ReLU[3] or ELU[4] have been proposed to address issues like dying neurons and to improve learning dynamics in deep networks (Goodfellow et al., 2016).

1.1.3 Pooling Layers

Pooling layers reduce the spatial size of feature maps, thus decreasing the number of parameters and computation in the network. Max pooling and average pooling are the two most common types. Max pooling returns the maximum value from the feature map patch, whereas average pooling computes the average. These layers contribute to translation invariance and help prevent overfitting by providing a form of local down sampling (Goodfellow et al., 2016).

1.1.4 Batch Normalization and Dropout

Batch normalization[5] normalizes the output of a layer by adjusting and scaling the activations. Introduced by Ioffe and Szegedy (2015), this technique accelerates training, improves performance, and provides some regularization. It works by ensuring that the distribution of inputs to each layer remains consistent throughout training. Overfitting is a major challenge in deep learning. Techniques such as dropout[6], which randomly deactivates neurons during training and batch normalization are commonly used to improve generalization and speed up convergence (Srivastava et al., 2014).

1.1.5 Fully Connected Layers

The fully connected layer[7], also referred to as a dense layer, is typically found toward the end of a CNN. Its purpose is to integrate the high-level features extracted by earlier layers (convolutional and pooling) and to perform the final decision-making step, often for classification or regression tasks. Each neuron is connected to every neuron

---

[2] https://www.tensorflow.org/api_docs/python/tf/keras/layers/ReLU
[3] https://www.tensorflow.org/api_docs/python/tf/keras/layers/LeakyReLU
[4] https://www.tensorflow.org/api_docs/python/tf/keras/layers/ELU
[5] https://www.tensorflow.org/api_docs/python/tf/keras/layers/BatchNormalization
[6] https://www.tensorflow.org/api_docs/python/tf/keras/layers/Dropout
[7] https://www.tensorflow.org/api_docs/python/tf/keras/layers/Dense

in the previous layer. This dense connectivity allows the network to consider all possible combinations of the extracted features (Goodfellow et al., 2016).

1.2 Design Considerations: Custom Models vs. Pretrained Models

Designing a CNN from scratch allows for architecture tailoring to specific data characteristics. However, it requires extensive experimentation and tuning of hyperparameters, including filter sizes, number of layers, learning rates, and regularization strategies. Custom models may outperform generic architectures when trained with sufficient data and optimized for the specific task.

The representational power of CNNs is influenced by the network's depth and width. Deeper networks can learn more abstract representations but may suffer from vanishing gradients and increased computational cost (Goodfellow et al., 2016).

Pretrained CNNs, including architectures like VGG (Simonyan & Zisserman, 2015), ResNet (He et al., 2016), Inception (Szegedy et al., 2015), MobileNet (Howard et al., 2017), and EfficientNet (Tan & Le, 2020), are typically trained on large-scale datasets such as ImageNet (Deng et al., 2009). Transfer learning involves either using these models as fixed feature extractors or fine-tuning them for the target task. Fine-tuning allows the model to adapt to domain-specific features while leveraging previously learned representations, resulting in improved performance and faster convergence on smaller datasets (Kornblith et al., 2019).

1.2.1 InceptionV3

InceptionV3[8] is an advanced version of the Inception architecture introduced by Szegedy et al. (2015), designed to improve both the computational efficiency and accuracy of convolutional neural networks. One of the key innovations in InceptionV3 is the use of factorized convolutions, which decompose large convolutions into smaller, more efficient ones, significantly reducing computational costs without sacrificing model performance (Szegedy et al., 2015). The architecture also features auxiliary classifiers, which are additional classifiers added to intermediate layers, helping to mitigate the vanishing gradient problem and improving the training process.

The depth and flexibility of InceptionV3 make it particularly effective for tasks that require high accuracy on large datasets. InceptionV3 has been widely used in a variety of domains, including medical imaging (Esteva et al., 2017) and object detection, where its ability to efficiently learn from large-scale data has been highly beneficial.

[8] https://www.tensorflow.org/api_docs/python/tf/keras/applications/InceptionV3?hl=en

1.2.2 VGG16

VGG16[9] is a deep convolutional neural network model introduced by Simonyan and Zisserman (2015) that has become one of the most well-known and widely used architectures for image classification tasks. The network architecture is distinguished by its simplicity and depth, using small 3x3 convolutional filters stacked on top of each other. VGG16 consists of 16 weight layers, including 13 convolutional layers and 3 fully connected layers. This depth allows the network to learn highly abstract features of images, which makes it effective for a wide variety of visual recognition tasks.

Despite its relatively simple architecture, VGG16 has shown remarkable performance on benchmark datasets such as ImageNet (Russakovsky et al., 2015). One of the primary reasons for VGG16's success is its ability to capture complex hierarchical patterns in images using a consistent structure of convolutional layers. However, the model is also known for its high computational cost, particularly due to the large number of parameters in the fully connected layers, which makes it less efficient compared to more recent architectures such as ResNet or EfficientNet.

[9] https://www.tensorflow.org/api_docs/python/tf/keras/applications/VGG16?hl=en

2 Efficient Deep Learning Techniques

As deep learning models grow in complexity and size, deploying them in real-world environments with limited computational resources becomes challenging. Efficient deep learning techniques aim to optimize neural networks for speed, memory, and energy consumption without significantly compromising accuracy. This chapter discusses several such techniques with a focus on pruning, quantization, and knowledge distillation. It also briefly outlines other methods such as neural architecture search and compact architectures.

2.1 Conceptual Framework for Efficient Deep Learning

Menghani (2021) outlines a structured approach to understanding the diverse methods aimed at improving the efficiency of deep learning systems. This framework categorizes techniques into five main domains - four pertaining to model optimization and one focused on supporting infrastructure.

- **Compression Techniques**

  These methods aim to reduce model size and computational load by compressing neural network layers. A prominent example is quantization, which reduces the numerical precision of model weights, which leads to decrease of memory usage and improvement of inference speed with minimal impact on performance.

- **Learning Techniques**

  These techniques involve modifications to the training process to yield more effective models that may require fewer resources. One such method is knowledge distillation, where a smaller model is trained to replicate the outputs of a larger, more accurate model.

- **Automated Techniques**

  Automated techniques such as hyperparameter optimization and neural architecture search (NAS) fall under this category. These methods systematically explore configuration and architecture spaces to identify models that achieve optimal trade-offs between accuracy and efficiency, including factors like latency and model size.

- **Efficient Neural Network Architectures**

  Innovations like convolutional layers and attention mechanisms represent major advances over traditional fully connected layers. These architectures enable

parameter sharing and more effective information flow, improving both scalability and robustness.

- **Infrastructure and Tooling**

  Efficient model deployment depends on supportive ecosystems such as TensorFlow[10], PyTorch[11], and their lightweight variants. These platforms offer the necessary infrastructure to realize the benefits of efficient model designs in real-world applications, particularly through support for quantized operations and optimized inference engines.

This framework provides a comprehensive lens through which the field of efficient deep learning can be understood, evaluated, and advanced (Menghani, 2021).

2.2 Model Compression Techniques

Model compression aims to reduce the size and complexity of neural networks, enabling faster inference and lower storage requirements.

2.2.1 Quantization

Quantization reduces the precision of weights and activations, typically from 32-bit floating-point to 8-bit integers, thereby decreasing model size and accelerating computation (Clark, 2024). This technique is effective in maintaining model accuracy while enhancing efficiency and is used in various domains.

Quantization is a fundamental technique employed in deep learning to optimize neural network models by reducing the numerical precision of parameters and computations. Instead of utilizing 32-bit floating-point representations (commonly denoted as FP32), quantization maps continuous high-precision values to lower-precision formats such as 16-bit floating-point (FP16), 8-bit integers (INT8), or even lower (Hugging Face, n.d.). The primary objective of quantization is to decrease model size, accelerate inference, and reduce energy consumption, making deep learning models more practical for deployment in resource-constrained environments such as mobile devices, embedded systems, and edge computing platforms (Clark, 2024).

In deep learning, quantization typically targets two main elements - weights (the learned parameters of the model) and activations (the intermediate outputs during inference) (PyTorch, 2019).

[10] https://www.tensorflow.org/
[11] https://pytorch.org/

Post-Training Quantization (PTQ)

Post-Training Quantization (PTQ)[12] is a widely used model compression technique designed to optimize deep neural networks for efficient deployment on resource-constrained devices. It involves converting a fully trained model, typically in 32-bit floating-point (FP32) precision, into a lower-precision format such as 8-bit integer (INT8) or 16-bit floating-point (FP16), without requiring retraining (Jacob et al., 2018). This reduction in numerical precision significantly decreases model size and inference latency, while also reducing power consumption—a critical consideration for mobile and edge devices (TensorFlow Model Optimization Team, 2019).

The quantization process maps continuous FP32 values of model weights and activations into a discrete integer space using quantization parameters, typically including a scale factor and a zero-point. A small, representative calibration dataset is used to estimate the distribution of activation values and to determine appropriate quantization ranges (Krishnamoorthi, 2018). This calibration step is essential to minimize quantization error and maintain the accuracy of the original model as closely as possible (Krishnamoorthi, 2018).

While PTQ offers substantial improvements in computational efficiency and memory usage, it can introduce a trade-off in model accuracy. The degree of degradation depends on the architecture and the distribution of weights and activations, some models are more robust to quantization than others. Nonetheless, PTQ remains highly advantageous when retraining is infeasible due to time, computational costs, or data privacy constraints. It is supported by major deep learning frameworks including TFLite[13] and PyTorch (Chenna, 2024).

Quantization-Aware Training (QAT)

Quantization effects are simulated during training, allowing the model to adapt to lower precision representations (Lang et al., 2024). This technique often preserves or even improves the original model's accuracy compared to PTQ (Cao & Aref, 2025).

Quantization-Aware Training (QAT)[14] is an advanced model optimization technique that integrates quantization directly into the training process of neural networks. Unlike PTQ, which applies quantization after training, QAT simulates the effects of low-precision arithmetic (such as 8-bit integers) during the forward and backward passes of training. This allows the model to learn to compensate for the numerical errors introduced by quantization, resulting in significantly higher accuracy retention compared to PTQ, especially for complex or sensitive models (Ray, 2024; Gholami et al., 2021).

---

[12] https://www.tensorflow.org/model_optimization/guide/quantization/post_training
[13] https://ai.google.dev/edge/litert
[14] https://www.tensorflow.org/model_optimization/guide/quantization/training_example

During QAT, the network is trained with "fake quantization" modules inserted into the computation graph. These modules simulate the effects of quantization (e.g., rounding and clipping to INT8 ranges) while maintaining full-precision weights for gradient updates. This dual representation enables the model to adapt to the constraints of reduced precision, effectively learning quantization-friendly parameters. The final trained model can then be exported with actual quantized weights for efficient inference on compatible hardware (Ray, 2024; Or et al., 2024).

QAT typically requires a longer training time and more computational resources compared to PTQ, as it involves retraining or fine-tuning the model with quantization effects simulated during training. However, it offers superior performance in terms of accuracy, particularly for networks with layers or operations that are highly sensitive to precision loss, such as depth wise convolutions or attention mechanisms in transformer-based architectures (Young, 2025). QAT is particularly beneficial when deploying models to edge devices, mobile platforms, and custom accelerators that support integer arithmetic.

Dynamic Quantization

Weights are statically quantized, but activations are quantized dynamically during inference based on observed input ranges (Gholami et al., 2021).

Dynamic quantization is a model optimization technique that converts a pre-trained neural network from full-precision (typically FP32) to lower-precision representations, such as INT8, with minimal intervention and no retraining required. Unlike PTQ, which statically quantizes both weights and activations ahead of inference, dynamic quantization defers the quantization of activations until runtime, where they are quantized dynamically based on the actual input data. This approach is particularly useful for models with highly variable activation distributions, where pre-computed quantization ranges may not generalize well across inputs (Hugging Face, n.d.).

In TensorFlow, this functionality is provided through the converter option called default. By setting the converter's optimization attribute to *tf.lite.Optimize.DEFAULT*, the model undergoes dynamic quantization (Google AI Edge, 2024b).

Performance Insights

In NVIDIA's TAO Toolkit[15], QAT significantly outperforms PTQ in maintaining model accuracy after quantization. In terms of inference performance on NVIDIA T4[16] GPUs, INT8 models trained with QAT nearly double the frames per second (FPS) compared to their FP16

[15] https://developer.nvidia.com/tao-toolkit
[16] https://www.nvidia.com/en-us/data-center/tesla-t4/

counterparts. PeopleNet[17]-ResNet18 achieves 762 FPS in FP16 and 1,517 FPS in INT8. The ResNet34 variant improves from 513 FPS (FP16) to 1,038 FPS (INT8) (Praveen, 2020).

TensorFlow's QAT approach demonstrates minimal accuracy degradation post-quantization. For example, the MobileNetV1[18] 224 model has a top-1 accuracy of 71.03% in its non-quantized form and 71.06% after 8-bit quantization. ResNetV1[19] 50 slightly decreases from 76.3% to 76.1%, and MobileNetV2[20] 224 from 70.77% to 70.01% (TensorFlow Model Optimization, n.d.-b).

In a practical example using the MNIST dataset, a Keras model achieved a baseline test accuracy of 95.51%. After applying QAT, the accuracy slightly increased to 95.84%, indicating that QAT can maintain or even improve accuracy in some cases (TensorFlow Model Optimization, n.d.-c).

Both NVIDIA's TAO Toolkit and TensorFlow's Model Optimization Toolkit show that QAT is effective in preserving model accuracy after quantization, often outperforming PTQ. Additionally, QAT enables significant improvements in inference performance, especially on hardware optimized for INT8 computations.

2.2.2 Pruning

Pruning is a model compression method that reduces the size and computational requirements of deep neural networks by removing redundant or less important parameters, such as weights, neurons, or filters. This technique enables faster inference and makes it easier to deploy models on devices with limited resources, often with minimal impact on accuracy (Menghani & Singh, n.d.).

Unstructured Pruning

Unstructured pruning is a model compression technique that aims to reduce the computational and memory footprint of deep neural networks by eliminating individual weights that contribute least to model performance (Laurent et al., 2020). Unstructured pruning operates at the level of individual weight elements, yielding sparse weight matrices. This fine-grained approach provides higher theoretical compression rates and model size reduction. While this method can significantly reduce the number of parameters, it often requires specialized hardware or software to efficiently handle the resulting irregular structures (Cheng et al., 2024).

---

[17] https://catalog.ngc.nvidia.com/orgs/nvidia/teams/tao/models/peoplenet
[18] https://www.tensorflow.org/api_docs/python/tf/keras/applications/MobileNet
[19] https://www.tensorflow.org/api_docs/python/tf/keras/applications/ResNet50
[20] https://www.tensorflow.org/api_docs/python/tf/keras/applications/MobileNetV2

Structured Pruning

Structured pruning eliminates entire structures within the network, such as filters, channels, or layers. This approach maintains regularity in the network architecture, making it more compatible with standard hardware and often leading to actual speedups in inference (Gholami et al., 2021).

Semi-Structured Pruning

Semi-structured pruning strikes a balance between unstructured and structured pruning by removing patterns of weights (e.g., blocks or groups) within the network (Cheng et al., 2024). This method aims to combine the flexibility of unstructured pruning with the hardware efficiency of structured pruning.

Hyperparameters in Pruning

Pruning Ratio (Sparsity Level)

This is one of the most important hyperparameters. It defines the percentage of weights or structures to be removed from the network (Liu et al., 2019).

- Too high a ratio may lead to underfitting and a sharp drop in accuracy (Neo, 2024).
- Too low a ratio may not significantly reduce model complexity.
- Typical values: 10%–90%, depending on task and architecture.

Pruning Frequency

This defines how often pruning is applied during training.

- Frequent pruning can destabilize learning.
- Less frequent pruning may miss the opportunity to remove redundant weights.
- A common strategy is gradual pruning, where sparsity is increased step-by-step throughout training (Cheng et al., 2024).

Performance Insights

Han et al. (2015) applied weight pruning to the LeNet-5 (LeCun et al., 1995) model trained on the MNIST dataset, which consists of 28×28 grayscale images of handwritten digits. The number of parameters reduced from approximately 431 000 to 36 000 (reduction over 90%) without compromising accuracy. On the ImageNet dataset, Han et al. (2015) demonstrated that their pruning method reduced AlexNet's[21] parameters by 9 times and VGG-16's by 13 times without any degradation in accuracy.

[21] https://docs.pytorch.org/vision/main/models/generated/torchvision.models.alexnet.html

2.3 Learning Techniques

Optimizing the training process can lead to more efficient models that converge faster and generalize better.

2.3.1 Knowledge Distillation

First introduced by Hinton et al. (2015), knowledge distillation works by transferring the soft probabilistic outputs of the teacher model to the student model, enabling the latter to achieve comparable performance with reduced complexity. These soft labels contain information about class similarities and decision boundaries, allowing the student model to generalize better than if it were trained solely on the original labelled data.

During the distillation process, the student is trained to mimic the teacher's softened output distributions, which are generated by applying a temperature scaling factor to the logits in the softmax[22] function. A higher temperature reveals more nuanced inter-class relationships by flattening the softmax output. The student model is then optimized using a combined loss function that includes the standard cross-entropy loss with the true labels and a distillation loss based on the divergence between the student's and teacher's softened outputs (Hinton et al., 2015).

Key Hyperparameters in Knowledge Distillation

Temperature (T)

The temperature parameter smooths the output probabilities of the teacher model, providing more informative gradients for the student. A higher temperature results in a softer probability distribution (Hinton et al., 2015).

Loss Weighting Factor (α)

This factor balances the contribution between the distillation loss (from the teacher's soft targets) and the standard loss (from the true labels) (Borup, 2020). Proper tuning of α is essential for effective knowledge transfer.

Learning Rate and Optimization Strategy

The choice of learning rate and optimization algorithm can significantly impact the distillation process. Adaptive learning rates and optimizers like Adam[23] are commonly used to facilitate convergence.

---

[22] https://www.tensorflow.org/api_docs/python/tf/keras/activations/softmax
[23] https://www.tensorflow.org/api_docs/python/tf/keras/optimizers/Adam

Performance Insights

Chen et al. (2018) proposed KDFM, a knowledge distillation method using feature maps, and applied it to image classification tasks on dataset CIFAR-100[24]. Using DenseNet-100 (Huang et al., 2017) as the teacher and MobileNet as the student, the method achieved a reduction in model size of over 50% and an inference speedup by 2 to 6 times. The student model maintained comparable accuracy, with less than a 1% drop compared to the teacher model.

Khan et al. (2022) introduced DSNet, a compact student model distilled from a ResNet-50[25] teacher for melanoma detection. The number of parameters was reduced from over 23 million to just 0.26 million (a compression factor of 15×), while achieving a high classification accuracy of 91.7%. Inference time dropped from 14.55 seconds for the teacher to 2.57 seconds for the student.

2.4 Automated Techniques

Automating the design of neural network architectures can yield models that are both efficient and high-performing.

2.4.1 Hyperparameter Optimization

Hyperparameter optimization is the process of finding the best settings (hyperparameters) for training a neural network. These are values that control how the learning process works—like the learning rate, batch size, number of layers, number of epochs or dropout rate— but they are not learned by the model itself. Choosing the right hyperparameters can make a big difference in how well a model performs. Instead of guessing or using trial and error, hyperparameter optimization uses methods like grid search, random search or Bayesian optimization to systematically test different combinations and find the most effective ones. The goal is to improve the model's accuracy, training speed, and generalization ability without manually tuning everything (Yu & Zhu, 2020; Belcic & Stryker, 2024).

2.4.2 Neural Architecture Search (NAS)

Neural Architecture Search (NAS) is a method used to automatically design neural network architectures instead of relying on manual trial-and-error approaches. It works by exploring a large space of possible network designs such as different combinations of layers, filter sizes, and connections, and identifying the best-performing models for a given task. NAS typically uses algorithms like reinforcement learning, evolutionary algorithms,

[24] https://www.tensorflow.org/datasets/catalog/cifar100
[25] https://www.tensorflow.org/api_docs/python/tf/keras/applications/ResNet50

or gradient-based methods to search and optimize architectures based on performance metrics like accuracy, speed, or memory usage. The main goal of NAS is to create models that are both effective and efficient, making them especially useful for deployment on devices with limited computing resources (White et al., 2023).

2.5 Efficient Neural Network Architectures

Efficient neural network architectures are designed to optimize performance while minimizing computational resources, making them ideal for deployment in environments with limited processing power, such as mobile devices or embedded systems. These architectures achieve efficiency through innovative design strategies that reduce model size, computational complexity, and energy consumption without significantly compromising accuracy. (Guo et al., 2021).

EfficientNet

EfficientNet (Tan & Le, 2020) developed by Google AI introduces a compound scaling method that uniformly scales network depth, width, and resolution using a set of fixed scaling coefficients. This approach allows for a balanced increase in model capacity, leading to improved accuracy and efficiency. The EfficientNet family, ranging from B0 to B7, demonstrates that carefully balancing these dimensions can lead to better performance than arbitrarily scaling any single dimension (Agarwal, 2020).

MobileNet

Also from Google, MobileNet (Howard et al., 2017) architectures are tailored for mobile and embedded vision applications. They utilize depth wise separable convolutions, which factorize a standard convolution into a depth wise convolution and a pointwise convolution, significantly reducing the number of parameters and computational cost. MobileNetV2 introduces inverted residuals and linear bottlenecks, further enhancing efficiency (Singh, 2024).

2.6 Deployment Frameworks and Infrastructure

Efficient deployment of deep learning models requires supportive infrastructure and tools.

TensorFlow Lite

TensorFlow Lite (TFLite) is designed for deploying models on mobile and embedded devices, offering optimizations like quantization and operator fusion to enhance performance (TensorFlow Lite team, 2021).

PyTorch Mobile and ExecuTorch

PyTorch Mobile enables the deployment of PyTorch models on mobile platforms, providing tools for model optimization and compatibility with mobile hardware (PyTorch Mobile – PyTorch, n.d.).

ExecuTorch is a newer runtime as part of the PyTorch Edge ecosystem. It's designed for ultra-efficient on-device inference, especially on resource-constrained edge devices like microcontrollers, wearables, and low-power embedded systems (PyTorch ExecuTorch, n.d.).

3 Designs of Experiments

This chapter outlines the experimental framework designed to systematically evaluate the efficiency of selected efficient deep learning techniques for convolutional networks on classification tasks.

3.1 Research Questions

The aim of this thesis is to first select suitable classification datasets and design experiments to compare the effectiveness of various efficient deep learning techniques applied to convolutional neural networks (CNNs). The experiments aim to evaluate and compare the impact of techniques such as weight pruning, quantization, and knowledge distillation on model performance. In this context, the thesis will address the following research questions:

1. **RQ1: How does weight pruning affect model accuracy, size, and inference time for different convolutional architectures and datasets?**

   This question investigates the impact of weight pruning on CNNs, specifically examining whether pruning influences model accuracy, size, and inference time in varying ways depending on the convolutional architecture used. The analysis will explore trade-offs between model compression and performance in diverse experimental setups.

2. **RQ2: What is the impact of quantization on model performance for different convolutional architectures and datasets?**

   The intention is to explore how the quantization of model weights and activations influences performance metrics such as accuracy, size, and speed across different convolutional architectures and datasets. The goal is to understand the effectiveness of quantization in reducing model size while maintaining competitive performance on various tasks and data distributions.

3. **RQ3: Can knowledge distillation successfully transfer knowledge from a larger teacher model to a smaller student model while improving classification accuracy?**

   The question examines whether knowledge distillation can effectively improve the performance of smaller CNN models by transferring knowledge from a larger, more complex teacher model. The focus will be on assessing the success of this technique across different convolutional architectures and datasets, determining whether the student model can achieve improved classification accuracy in comparison to its baseline performance.

3.2 Methodology

The methodology follows a systematic experimental approach, ensuring rigorous evaluation of each technique under controlled conditions. This chapter describes the essential components of the experimental framework used to compare selected efficient deep learning techniques for convolutional networks on classification tasks. These components include efficient deep learning techniques selection, dataset selection, model selection, training procedures, application of efficiency techniques, evaluation metrics, and implementation environment.

Efficient Deep Learning Techniques Selection

Before applying efficient deep learning methods in practice, an initial research phase was conducted to identify techniques most relevant to improving the performance of convolutional neural networks (CNNs) in classification tasks. This process is outlined in detail in Chapter 2.

Based on the literature and current trends in deep learning optimization, three techniques were selected for their balance of practicality, popularity, and performance impact:

- Pruning was chosen for its ability to reduce model size and computation by eliminating less significant weights.
- Knowledge distillation was elected as a way to retain high model performance in a smaller network by transferring knowledge from a larger teacher model.
- Quantization was included for its effectiveness in reducing model size and accelerating inference through lower-precision computations.

These techniques were prioritized due to their strong support in mainstream deep learning frameworks (e.g., TensorFlow), their frequent use in real-world applications, and the availability of relevant tools and documentation. Together, they provide a representative and complementary approach to enhancing model efficiency in the context of this thesis.

Datasets Selection

For this thesis, two datasets were selected to ensure diversity in domain, number of classes, and the number of training examples:

- TF Flowers[26] - A dataset consisting of images of different flower species, used to evaluate the performance of models on a relatively simpler classification task.

[26] https://www.tensorflow.org/datasets/catalog/tf_flowers

- Stanford Dogs[27] - A dataset containing images of different dog breeds, representing a more complex classification task due to the high intra-class variability.

Model Selection

The experiments involves two different modelling approaches:

- A custom convolutional neural network (CNN) trained from scratch on one dataset.
- A pretrained model applied to another dataset using transfer learning.

Training Procedure

The training procedure follows a structured and standardized approach to ensure consistency and reliability across all experiments. It encompasses data preprocessing, model initialization, hyperparameter selection, and optimization strategies tailored to the chosen convolutional networks. The key steps in the training process are as follows:

- **Data Preprocessing**
  Before training, the datasets go through standard preprocessing steps, including normalization and resizing to a fixed input shape suitable for the models. The dataset trained on the pretrained model was pre-processed using *inception_v3.preprocess_input function*, *mobilenet_v2.preprocess_input* or *vgg16.preprocess_input* from *tensorflow.keras.applications*[28] depending on the model.
- **Model Initialization**
  Two distinct training paradigms are followed:
  - Training a custom CNN from scratch on one dataset.
  - Fine-tuning a pre-trained CNN on another dataset using transfer learning.
- **Hyperparameter Selection**
  To optimize performance, hyperparameters such as learning rate, batch size, and regularization techniques are selected based on preliminary experiments.
- **Optimization Strategy**
  The Adam optimizer[29] is primarily used due to its adaptive moment estimation capabilities, enhancing convergence speed and stability. A categorical cross-entropy loss function is utilized, as it is well-suited for multi-class classification tasks.
- **Early Stopping**[30]
  To prevent overfitting, early stopping is implemented based on validation loss.

---

[27] https://www.tensorflow.org/datasets/catalog/stanford_dogs
[28] https://www.tensorflow.org/api_docs/python/tf/keras/applications
[29] https://www.tensorflow.org/api_docs/python/tf/keras/optimizers/Adam
[30] https://www.tensorflow.org/api_docs/python/tf/keras/callbacks/EarlyStopping

Evaluation Metrics

To objectively evaluate and compare the performance of the selected efficient deep learning techniques, a set of standard metrics is employed. These metrics measure not only classification performance but also the efficiency of the models in terms of speed and resource usage. The three primary metrics considered in this thesis are:

- **Accuracy**
  Accuracy is the proportion of correctly predicted labels over the total number of predictions. It is one of the most commonly used metrics in classification tasks (Goodfellow et al., 2016). Accuracy is formally defined as:

$$Accuracy = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}} \tag{3.1}$$

- **Inference Time**
  Inference time refers to the time it takes for a trained model to make a prediction on new input data. This metric is crucial for evaluating the practical deploy ability of a model, especially in real-time or resource-constrained environments (Dominguez, 2025). Lower inference time indicates a faster and more efficient model.
- **Model Size**
  Model size refers to the total storage space required by the trained model, usually measured in megabytes (MB). This metric is closely linked to memory footprint and affects the model's feasibility for deployment on devices with limited storage and RAM (Han et al., 2016). Techniques such as pruning and quantization are specifically designed to reduce model size without significantly sacrificing accuracy.

Implementation Environment

The experiments were conducted using TensorFlow (Abadi et al., 2015) and Keras (Chollet et al., 2015) on Google Colab (Bisong, 2019) to ensure optimal training and inference conditions. All models and experiments were implemented using Python.

3.3 Datasets Selection

For the experiments in this bachelor's thesis, the selection of appropriate datasets was a critical task in evaluating and comparing the performance of various efficient deep learning techniques for convolutional networks on classification tasks. Two primary image datasets were initially considered to explore the various aspects of the classification problem – Places365 Small[31] and TF Flowers. The use of distinct datasets was intentional, as it allowed for the examination of model performance across varying image characteristics

[31] https://www.tensorflow.org/datasets/catalog/places365_small

and categories, ensuring a comprehensive analysis of the classification and efficient techniques in different contexts.

Places365 Small (Zhou et al., 2017) focuses on a wide variety of visual scenes, ranging from natural landscapes to urban environments, and includes highly diverse visual contexts. Itcomprises 365 classes, making it a challenging classification task. This dataset was selected to assess how well the model could handle a large number of classes and complex visual features. Given its vast scale, Places365 Small offered an excellent opportunity to test the scalability and robustness of convolutional networks when efficient techniques were applied. However, despite its potential, the large number of classes and high complexity proved to be a significant challenge. The size of the dataset made it difficult to experiment effectively and efficiently, as training times were long and managing the complexity became hard.

TF Flowers (The TensorFlow Team, 2019), on the other hand, was selected as a more focused dataset, being centred on one type of object. This dataset consisted of only five classes, and so it was possible to attempt models that were fine-tuned for a lower class number while still requiring sensitivity to finer visual details, such as shapes and colours. Even in its reduced size, the complex visual features of each flower class were still an intriguing challenge to classify for models. The relatively small size of the data allowed for experimentation and testing of different model versions without the need for extended training times.

Another dataset, Patch Camelyon[32] (Veeling et al., 2018), was also considered for its binary classification task from histopathological images. It is a dataset that demands detecting tiny visual characteristics in small image sizes (96x96 pixels) with greater attention to details. Being a smaller set with fewer classes and demanding attention to detail, it was a good candidate for exploring efficient deep learning techniques in the classification of medical images.

After conducting preliminary experiments with Places365 Small, it became clear that the dataset's sheer scale and complexity hindered the efficiency of the experimentation process. The vast number of classes and the significant variation in visual contexts made it a difficult dataset to manage for this thesis, leading to its eventual replacement. In place of Places365 Small, the Stanford Dogs dataset was chosen for further experiments. This dataset offers a more manageable classification task, focused on distinguishing between different breeds of dogs. While it still requires attention to fine-grained visual details, the reduced number of classes and more focused scope provided a better balance between complexity and experimental feasibility.

Stanford Dogs (Khosla et al., 2011) contains 120 classes, allowing for effective testing and comparison of efficient deep learning techniques in convolutional networks, while still providing enough challenge to demonstrate the practical effectiveness of the models. This dataset was ultimately chosen for the final experiments, as it provided a realistic yet

[32] https://www.tensorflow.org/datasets/catalog/patch_camelyon

computationally feasible balance ideal for testing and comparison of the various efficient deep learning approaches used by this thesis.

Tab. 3.1 Comparison of TF Flowers and Stanford Dogs (Source: Author)

Feature	**TF Flowers**	**Stanford Dogs**
Number of Classes	5 (flower species)	120 (dog breeds)
Number of Images	3 670 images	20 580 images
Dataset Size	Small	Large
Class Variability	Moderate (5 species with clear visual differences)	High (120 breeds with subtle differences)
Visual Challenge	Requires recognizing detailed features such as petal shapes and colour	Requires distinguishing between breeds based on subtle features like fur texture, size, shape, and posture
Task Difficulty	Moderate (5 classes make it manageable, but fine-grained features remain challenging)	High (many classes with subtle differences)
Experimentation Suitability	Quick experimentation with models, ideal for testing basic to intermediate deep learning techniques	Better for testing more advanced techniques with larger datasets and the need for fine-tuning
Tensorflow Link	tf_flowers[33]	stanford_dogs[34]

3.4 CNN Models

To establish a strong foundation for the experiments, two baseline models were created — one for the TF Flowers dataset and another for the Stanford Dogs dataset. These baseline models serve as reference architectures to evaluate different efficient deep learning techniques. While minor modifications may be applied during specific experiments, the general structure of these models remains consistent throughout the experiments.

[33] https://www.tensorflow.org/datasets/catalog/tf_flowers
[34] https://www.tensorflow.org/datasets/catalog/stanford_dogs

3.4.1 Baseline Model for TF Flowers Dataset

The first baseline model[35] is a custom convolutional neural network (CNN) designed for the TF Flowers dataset. The architecture consists of multiple convolutional layers followed by batch normalization, activation functions, and max pooling. The model progressively increases the number of filters (Code 3.1) while reducing the spatial dimensions, ending with a fully connected dense layer for classification.

Code 3.1 Baseline Model for Dataset TF Flowers (Source: Author)

```
baseline_model = keras.Sequential([
    keras.layers.Conv2D(16, (3, 3), use_bias=False,
input_shape=(IMG_SIZE, IMG_SIZE, 3)),
    keras.layers.BatchNormalization(),
    keras.layers.Activation('relu'),
    keras.layers.MaxPooling2D((2, 2)),

    keras.layers.Conv2D(32, (3, 3), use_bias=False),
    keras.layers.BatchNormalization(),
    keras.layers.Activation('relu'),
    keras.layers.MaxPooling2D((2, 2)),

    keras.layers.Conv2D(64, (3, 3), use_bias=False),
    keras.layers.BatchNormalization(),
    keras.layers.Activation('relu'),
    keras.layers.MaxPooling2D((2, 2)),

    keras.layers.Conv2D(128, (3, 3), use_bias=False),
    keras.layers.BatchNormalization(),
    keras.layers.Activation('relu'),
    keras.layers.MaxPooling2D((2, 2)),

    keras.layers.Conv2D(256, (3, 3), use_bias=False),
    keras.layers.BatchNormalization(),
    keras.layers.Activation('relu'),
    keras.layers.MaxPooling2D((2, 2)),

    keras.layers.Flatten(),
    keras.layers.Dense(256, activation='relu'),
    keras.layers.Dropout(0.4),
    keras.layers.Dense(num_classes, activation='softmax')
])
```

[35] The corresponding model is available in *BASELINE_TF_FLOWERS*.ipynb in Appendix A.

Convolutional Blocks

- Five convolutional layers progressively increase the number of filters (from 16 to 256), allowing the model to capture increasingly complex features.
- Batch normalization is applied after each convolutional layer to stabilize training and speed up convergence.
- Max pooling is used after each activation to reduce spatial dimensions and limit overfitting.

Fully Connected Layers

- A 256-unit dense layer helps in feature extraction.
- A dropout layer (0.4) is applied to reduce overfitting.
- A softmax output layer ensures multi-class classification for the five flower species.

Training Details

- Batch Size: 64
- Image Size: 224x224
- Optimizer: Adam
- Number of Epochs: 30
- Patience of Early Stopping: 9
- Learning Rate: 0.002

Results for TF Flowers Dataset

- Test Accuracy: 75.56%
- Inference Time: 0.0741 s/image (on GPU T4)

3.4.2 Baseline Model for Stanford Dogs Dataset

For the Stanford Dogs dataset, a pretrained InceptionV3 is used as the baseline. The choice of InceptionV3 is justified by its efficiency in handling fine-grained image classification tasks and its ability to capture complex visual patterns.

A custom CNN model, different from the one used for TF Flowers, was initially developed for the Stanford Dogs dataset. This model followed a deeper architecture to handle the complexity of distinguishing between 120 dog breeds.

Despite increasing the number of convolutional layers and fine-tuning hyperparameters, the model failed to achieve a satisfactory level of test accuracy, which was around 15% after multiple training runs (30 epochs). The custom CNN struggled to differentiate between similar dog breeds, which require fine-grained feature extraction.

Due to these limitations, the custom CNN model was discarded, and a pretrained VGG16 model was initially considered. However, further experimentation revealed that InceptionV3 yielded better results in terms of test accuracy and computational efficiency. Consequently, VGG16 was replaced with InceptionV3 as the final baseline model[36] (Code 3.2).

Code 3.2 Baseline Model for Dataset Stanford Dogs (Source: Author)

```
base_model = InceptionV3(weights='imagenet', include_top=False,
input_shape=(img_size, img_size, 3))

for layer in base_model.layers:
    layer.trainable = False

model_inception = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(num_classes, activation='softmax')
])
```

Transfer Learning Approach

- InceptionV3, pre-trained on ImageNet, is used as a feature extractor.
- All convolutional layers are frozen to retain pre-trained feature representations.
- Only the top layers (classifiers) are trained, making the model computationally efficient.

Fully Connected Layers

- A global average pooling layer replaces the fully connected layers of InceptionV3, reducing the number of parameters.
- A 256-unit dense layer with ReLU activation helps extract features specific to dog breeds.
- A dropout layer (0.5) prevents overfitting.
- A softmax output layer provides predictions for the 120 dog breeds.

Training Details

- Batch Size: 32
- Image Size: 299x299
- Optimizer: Adam

[36] The corresponding model is available in *BASELINE_STANFORD_DOGS.ipynb* in Appendix A.

- Number of Epochs: 10
- Patience of Early Stopping: 3
- Learning Rate: 0.0001
- Dataset preprocessing using the preprocess_input function from keras.applications based on the model.

Results for Stanford Dogs Dataset

- Test Accuracy: 90.95% (after training for 10 epochs with frozen layers)
- Inference Time: 0.1082 s/image (on GPU T4)

3.4.3 Final Baseline Models and Use in Experiments

Both baseline models serve as reference architectures in the experiments conducted further in this thesis. While the TF Flowers model is a lightweight custom CNN built from scratch, the Stanford Dogs baseline model leverages transfer learning with InceptionV3 for improved feature extraction.

The TF Flowers model (Tab. 3.2), with 3.67 million parameters and also 3.67 million trainable parameters, provides a strong baseline for evaluating various deep learning techniques in a smaller, simpler dataset. Due to its efficient structure, it enables quick experimentation with different optimizations, achieving a training time of approximately 7 seconds per epoch. Despite its relatively low complexity, it achieves 75% test accuracy, the model is relatively lightweight and allows for quick experimentation.

The Stanford Dogs baseline model demonstrates how pre-trained architectures can be used for fine-grained classification. It is expected to benefit from fine-tuning in later experiments, where deeper layers may be unfrozen for better performance.

In contrast, the Stanford Dogs baseline models (Tab. 3.2) illustrate the advantages of pre-trained architectures for fine-grained classification. The VGG16-based model, with 15.03 million parameters but only 324 216 trainable parameters, has the longest training time (74 seconds per epoch). This model was discarded and replaced by InceptionV3 due to lower accuracy. The InceptionV3-based model, with 22.36 million parameters and 555 384 trainable parameters, balances efficiency and accuracy, achieving 90.95% test accuracy while training in 62 seconds per epoch. Fine-tuning some deeper layers could improve test accuracy but would increase computational cost.

Both final baseline models will undergo modifications and optimizations throughout the experiments, such as changes in hyperparameters, additional regularization techniques, and improvements in training strategies. These experiments aim to assess the impact of different efficient techniques while maintaining computational efficiency.

Tab. 3.2 Comparison of the Two Baseline Models and discarded VGG16 Model (Source: Author)

Feature	TF Flowers Baseline Model (Custom CNN)	Stanford Dogs Baseline Model (InceptionV3)	Initial Stanford Dogs Model (VGG16)
Model Type	Custom CNN	Pretrained InceptionV3	Pretrained VGG16
Dataset	TF Flowers	Stanford Dogs	Stanford Dogs
Number of Parameters	3 673 973	22 358 168	15 038 904
Trainable Parameters	3 672 981	555 384	324 216
Training Time (per epoch)	7 seconds	62 seconds	74 seconds
Inference Time (s/image)	0.0741	0.1082	0.0941
Test Accuracy	75.56%	90.95%	64.72%
Train Accuracy	74.62%	89.29%	57.02%

3.5 Experiments

This section outlines the experimental setup and details for the evaluation of various efficient deep learning optimization techniques applied to convolutional neural networks (CNNs). The experiments primarily focus on weight pruning, quantization-aware training (QAT), and knowledge distillation, tested across different CNN architectures and datasets. Each technique will be assessed on how it impacts model accuracy, size, and inference time.

To ensure clarity and traceability between the described experiments and their practical implementation, Tab. 3.3 summarizes the relationship between individual experiments, their associated research questions (RQs), and the Jupyter Notebooks in which they were carried out. List of all Jupyter Notebooks are listed in *Appendix A – Source Code*.

Tab. 3.3 Overview of Experiments (Source: Author)

Experiment	RQ	Jupyter Notebook
Weight Pruning on Custom CNN	RQ1	TF_FLOWERS_pruning.ipynb

Quantization-Aware Training and Post-Training Quantization on Custom CNN	RQ2	TF_FLOWERS_quantization.ipynb
Knowledge Distillation on Custom CNN	RQ3	TF_FLOWERS_distillation.ipynb
Post-Training Quantization on Pretrained Model	RQ2	STANFORD_DOGS_quantization_ptq.ipynb
Quantization-Aware Training on Pretrained Model	RQ2	STANFORD_DOGS_quantization_qat.ipynb
Weight Pruning on Pretrained Model	RQ1	STANFORD_DOGS_pruning.ipynb

For the experiments, two datasets were used - TF Flowers and Stanford Dogs. Both datasets were loaded using TensorFlow Datasets (TFDS) and were consistently split into training, validation, and test subsets across all experiments to ensure comparability.

The Stanford Dogs dataset was divided by allocating 80% of the original training data for training, the remaining 20% for validation, and using the official test split as the test set.

The TF Flowers dataset, which contains only a single training split, was manually partitioned into 80% for training, 10% for validation, and 10% for testing.

In both cases, the data was loaded as image-label pairs and the files were shuffled to improve randomness during training.

3.5.1 Weight Pruning on Custom CNN

Dataset: TF Flowers

Model: Custom CNN

Description

Weight pruning is applied to a custom CNN model to reduce the number of parameters by eliminating weights based on their magnitude. This is an unstructured pruning method where weights with lower magnitudes are removed, leading to sparse weight matrices. While pruning is expected to reduce the model's size, the irregular sparsity patterns may hinder efficient hardware acceleration.

Main Expectation

The primary expectation is that weight pruning will lead to a significant reduction in model size, but with a slight trade-off in accuracy. Due to the sparsity introduced in the weight matrices, it is anticipated that inference time may not see a considerable improvement, and it could even be hindered by the irregularities in the sparse matrices.

Technique: Weight Pruning

Tested Hyperparameters

- The pruning hyperparameters were chosen through trial and error to investigate the effects of different sparsity schedules.
- The TensorFlow Model Optimization Guide was used as a reference, which suggests a default configuration with initial sparsity of 0.50 and final sparsity of 0.80 (TensorFlow Model Optimization, n.d.-a).
- In this thesis, initial sparsity values ranged from 0.0 to 0.7, and final sparsity values ranged from 0.3 to 0.9, while keeping the end step constant. This allowed evaluation of both mild and aggressive pruning strategies.

Metrics

- Accuracy
- Inference Time
- Model Size
- Non-Zero Parameters (Model Sparsity)

Training Configuration:

- Optimizer: Adam (learning rate = 0.002)
- Loss: Sparse Categorical Cross-Entropy
- Epochs: 30
- Batch Size: 64
- Preprocessing: Resize and rescale input images
- Evaluation Dataset: Test dataset

3.5.2 Quantization-Aware Training and Post-Training Quantization on Custom CNN

Dataset: TF Flowers

Model: Custom CNN

Description

Quantization-Aware Training (QAT) is applied to a custom CNN trained on the TF Flowers dataset. Two different configurations are evaluated:

- Quantization layers are applied to all trainable layers in the network
- Only the convolutional layers are quantized

QAT simulates the effects of quantization (e.g., int8 operations) during the forward and backward passes of training, enabling the model to learn representations that are more robust to quantization artifacts. After training, the models are converted to TFLite format for optimized deployment.

Post-training quantization (PTQ) is applied to a custom CNN model to reduce its computational and memory footprint without requiring quantization during training. This includes converting weights and activations to lower-precision formats such as float16 (FP16) and 8-bit integers (INT8). Additionally, the model is converted to TensorFlow Lite (TFLite) to enhance deployment efficiency. PTQ is a widely used method for compressing models after training, making it suitable for edge and mobile environments.

Main Expectation

QAT is expected to preserve accuracy better than post-training quantization, especially for lower bit-depth formats like int8. The goal is to reduce model size and improve inference time without significantly degrading classification performance. By strategically selecting which layers to quantize, the experiment explores trade-offs between model compression and accuracy.

The expectation is that PTQ will significantly reduce model size and improve inference time, with only a minor impact on accuracy. The experiment explores how different quantization strategies affect the trade-offs between accuracy and efficiency, particularly in resource-constrained scenarios.

Technique: Quantization-Aware Training (QAT) with TensorFlow Model Optimization Toolkit and Post-Training Quantization (TFLite Conversion, FP16, INT8)

Tested Configurations

- Quantizing all layers
- Quantizing only convolutional layers

Both versions are also converted to TFLite using int8 precision for comparison

- Baseline TFLite (default PTQ by TFLite converter)
- FP16 Quantization
- INT8 Quantization

Metrics

- Model Size
- Inference Time (s/image)
- Accuracy

Training Configuration:

- Optimizer: Adam (learning rate = 0.002)
- Loss: Sparse Categorical Cross-Entropy
- Epochs (baseline): 30
- Epochs (QAT): 15
- Batch Size: 64
- Preprocessing: Resize and rescale input images
- Evaluation Dataset Size: Test dataset

3.5.3 Knowledge Distillation on Custom CNN Using Pretrained Model

Dataset: TF Flowers

Models: A pretrained InceptionV3 model with fine-tuning was used as the teacher, while a custom CNN—modified by replacing the Flatten layer with a GlobalAveragePooling2D layer—served as the student.

Description
In this experiment, knowledge distillation is applied, where a smaller student model (Custom CNN) learns to mimic the predictions of a larger teacher model. The teacher model is pretrained, and the student model is trained to replicate the teacher's softened predictions using distillation. The goal is to assess whether knowledge distillation can improve the student model's generalization and accuracy over independent training.

Main Expectation

It is expected that knowledge distillation will improve the accuracy and generalization of the student model compared to training the student independently. By transferring knowledge from the teacher, the student should benefit from the teacher's higher-level abstractions, which should enhance performance, especially in terms of classification accuracy.

Technique: Knowledge Distillation

Tested Hyperparameters

- The hyperparameters were chosen through trial and error.
- The implementation of classical knowledge distillation from Borup (2020) was used as a reference, which suggests a default configuration with alpha of 0.10 and temperature of 3.
- Distillation Temperature: 1–20
- Alpha: Weight for Teacher-Student Loss (0.1–0.9)

Metrics

- Accuracy
- Inference Time
- Model Size

Training Configuration

- Optimizer: Adam
- Loss: Sparse Categorical Cross-Entropy
- Epochs (Distilled Model): 20
- Batch Size: 64
- Preprocessing: Resize and rescale input images
- Evaluation Dataset Size: Test dataset

3.5.4 Post-Training Quantization on Pretrained Model

Dataset: Stanford Dogs

Model: InceptionV3

Description

Post-training quantization (PTQ) is applied to a custom CNN model to reduce its computational and memory footprint without requiring quantization during training. This includes converting weights and activations to lower-precision formats such as float16 (FP16) and 8-bit integers (INT8). Additionally, the model is converted to TensorFlow Lite using dynamic range quantization to enhance deployment efficiency. PTQ is a widely used method for compressing models after training, making it suitable for edge and mobile environments.

Main Expectation

The expectation is that PTQ will significantly reduce model size and improve inference time, with only a minor impact on accuracy. The experiment explores how different quantization strategies affect the trade-offs between accuracy and efficiency, particularly in resource-constrained scenarios.

Technique: Post-Training Quantization

Tested Variants:

- Baseline TFLite (Default PTQ - dynamic range quantization)
- FP16 Quantization
- INT8 Quantization

Metrics

- Model Size
- Inference Time (s/image)
- Accuracy

Training Configuration

- Optimizer: Adam with learning rate 0.0001
- Loss: Sparse Categorical Cross-Entropy
- Epochs: 10
- Batch Size: 32
- Preprocessing: Resize and preprocessing function for InceptionV3 tf.keras.applications.inception_v3.preprocess_input
- Evaluation Dataset Size: 1000 samples from test dataset

3.5.5 Quantization-Aware Training on Pretrained Model

Dataset: Stanford Dogs

Models: InceptionV3, MobileNetV2

Description
Quantization-aware training (QAT) is applied to the pretrained InceptionV3 model with all parameters, with the goal of reducing the precision of the model's computations without sacrificing accuracy. This experiment aims to explore how QAT affects a deeper, pretrained model, using the Stanford Dogs dataset. The model's performance in terms of size reduction and inference time will be evaluated. The same will be done on MobileNetV2 for comparison.

Main Expectation

Similar to the custom CNN experiment, it is expected that QAT will reduce the model size and improve inference time while maintaining high accuracy. This will be tested on the Stanford Dogs dataset to evaluate how QAT impacts the performance of the more complex InceptionV3 model.

Technique: Quantization-Aware Training

Tested Hyperparameters

- Precision: INT8
- Number of Quantized Layers: All layers

Metrics

- Model Size Reduction
- Inference Time
- Accuracy

Training Configuration

- Optimizer: Adam with learning rate 0.0001
- Loss: Sparse Categorical Cross-Entropy
- Epochs: 7
- Batch Size: 32
- Preprocessing:
    - Resize and preprocessing function for InceptionV3 *tf.keras.applications.inception_v3.preprocess_input*
    - Resize and preprocessing function for MobileNetV2 *tensorflow.keras.applications.mobilenet_v2.preprocess_input*
- Evaluation Dataset Size: 1000 samples from test dataset

Environment

- This experiment was conducted using Google Colab with an NVIDIA A100 GPU and TensorFlow 2.14.1.

3.5.6 Weight Pruning on Pretrained Model

Dataset: Stanford Dogs

Model: InceptionV3

Description
Weight pruning is applied on a pretrained model InceptionV3 to reduce the number of parameters by eliminating weights based on their magnitude. This is an unstructured pruning method where weights with lower magnitudes are removed, leading to sparse weight matrices. While pruning is expected to reduce the model's size, the irregular sparsity patterns may hinder efficient hardware acceleration.

Main Expectation

The primary expectation is that weight pruning will lead to a significant reduction in model size, but with a slight trade-off in accuracy. Due to the sparsity introduced in the weight matrices, it is anticipated that inference time may not see a considerable improvement, and it could even be hindered by the irregularities in the sparse matrices.

Technique: Weight Pruning

Tested Hyperparameters

- The pruning hyperparameters were chosen through trial and error to investigate the effects of different sparsity schedules.
- The TensorFlow Model Optimization Guide was used as a reference, which suggests a default configuration with initial sparsity of 0.50 and final sparsity of 0.80 (TensorFlow Model Optimization, n.d.-a).
- In this thesis, initial sparsity values ranged from 0.1 to 0.7, and final sparsity values ranged from 0.3 to 0.9, while keeping the end step constant.

Metrics

- Accuracy
- Inference Time
- Model Size
- Non-zero Parameters (model sparsity)

Training Configuration

- Optimizer: Adam with learning rate 0.0001
- Loss: Sparse Categorical Cross-Entropy
- Epochs: 3
- Batch Size: 32
- Preprocessing: Resize and preprocessing function for InceptionV3 tf.keras.applications.inception_v3.preprocess_input
- Evaluation Dataset Size: Test dataset

Environment

- This experiment was conducted using Google Colab with an NVIDIA A100 GPU and TensorFlow 2.14.1.

3.6 Implementation Details

The experiments were conducted using Google Colab (Bisong, 2019), which provides cloud-based access to GPU resources. Unless otherwise specified, the NVIDIA T4 GPU was primarily used for training and inference across most experiments. In some cases, the NVIDIA A100 GPU was utilized to accelerate specific runs, particularly those involving more computationally intensive models or training procedures.

3.6.1 Libraries and Frameworks

The implementation was primarily based on TensorFlow and Keras, with additional libraries used for dataset management, model optimization, and performance evaluation. The key libraries included:

- TensorFlow (2.18.0, 2.14.1) and Keras were used as the core frameworks for building, training, and optimizing deep learning models. Unless otherwise specified, TensorFlow 2.18.0 was used by default.
- TensorFlow Datasets (4.9.8) facilitated access to standard benchmark datasets, including TF Flowers and Stanford Dogs.
- Scikit-learn (1.6.1), specifically the *model_selection* module, was employed for splitting datasets into training and validation subsets.
- TensorFlow Model Optimization Toolkit (0.8.0) played a critical role in implementing model compression techniques, including weight pruning and quantization-aware training.
- NumPy (1.26.4) was used for numerical operations and array manipulation throughout the pipeline.
- Pandas (v2.2.2) assisted in structured data handling and preprocessing and exploratory data analysis.
- Matplotlib (3.10.0) and Seaborn (0.13.2) supported the visualization of model performance metrics and dataset characteristics.
- OS and Time modules were used for managing file paths, handling system-level tasks, and measuring execution time.

4 Results of Experiments

4.1 Weight Pruning on Custom CNN

4.1.1 Objective

The purpose of this experiment is to assess the impact of Weight Pruning on the performance of a custom CNN model trained on the TensorFlow Flowers dataset. Weight pruning involves removing connections in the network based on their magnitude, thereby creating sparse weight matrices. This experiment evaluates how varying levels of sparsity affect the model's accuracy, size, inference time, and number of non-zero parameters.

4.1.2 Results

The detailed results are summarized in Tab. 4.1 and Tab. 4.2. In the following section, key observations and trends from these results are described and discussed.

Accuracy

- Aggressive pruning (0.3 - 0.9) improved test accuracy, achieving 80.65%, which as the only one outperforms the baseline by approximately 1.7%.
- Low-to-mid sparsity levels (20–50% remaining weights) generally preserved or even boosted accuracy.
- Heavy pruning (0.4 - 0.7) caused noticeable accuracy drop by 10.63%.

Inference Time

- Small improvements were observed in inference times across most pruned models.
- The fastest configuration was 0.3 - 0.9 with an inference time of 0.0516 s/image.
- However, due to unstructured pruning, extremely sparse models did not show major speedups.

Model Size and Non-Zero Parameters

- Model size before compression is same for all models, because the number of parameters did not change.
- Significant change of model size was achieved by using compression to gzip[37] format. Compressed model size dropped from 7.19 MB (baseline model) to just 1.30 MB (0.3 - 0.9), which is the smallest, and the number of nonzero parameters reduced by up to 90%.

[37] https://docs.fileformat.com/compression/gzip/

Tab. 4.1 Weight Pruning of Custom CNN – Accuracy (Source: Author)

Sparsity (Initial - Final)	Test Accuracy	Validation Accuracy	Train Accuracy	Accuracy Drop (from Baseline)	Average Inference Time (s/image)
Baseline Model	79.29%	75.2%	80.79%	-	0.0521
0.0 - 0.5	74.65%	77.38%	83.14%	-4.63%	0.0540
0.2 - 0.8	78.74%	71.66%	77,00%	-0.55%	0.0547
0.3 - 0.9	80.65%	74.93%	81.13%	1.36%	0.0516
0.3 - 0.5	78.47%	77.38%	85.96%	-0.82%	0.0538
0.0 - 0.7	77.11%	75.2%	81.5%	-2.18%	0.0545
0.1 - 0.3	71.38%	73.56%	68.29%	-7.90%	0.0542
0.1 - 0.5	74.65%	76.83%	86.85%	-4.63%	0.0536
0.4 - 0.7	68.66%	73.29%	70.36%	-10.63%	0.0536
0.5 - 0.8	76.56%	75.2%	80,00%	-2.72%	0.0553
0.7 - 0.9	75.75%	74.93%	78.09%	-3.54%	0.0533

Tab. 4.2 Weight Pruning of Custom CNN – Size (Source: Author)

Sparsity (Initial - Final)	Model Size (MB)	Gzipped Model Size (MB)	Nonzero parameters	Weight Percentage
Baseline model	7.82	7.19	2 034 037	100%
0.0 - 0.5	7.82	4.43	1 060 891	52%
0.2 - 0.8	7.82	2.20	434 056	21%
0.3 - 0.9	7.82	1.30	208 131	10%
0.3 - 0.5	7.82	4.38	1 035 243	51%
0.0 - 0.7	7.82	2.92	614 939	30%
0.1 - 0.3	7.82	5.58	1 424 578	70%

0.1 - 0.5	7.82	4.44	1 052 341	52%
0.4 - 0.7	7.82	3.02	657 036	32%
0.5 - 0.8	7.82	2.40	481 531	24%
0.7 - 0.9	7.82	1.29	206 326	10%

4.1.3 Findings

- Weight pruning can effectively shrink model size while maintaining, or even improving, performance, especially within the 20-50% weight range.
- Over-pruning without careful tuning can severely degrade model accuracy.
- Inference time gains were modest, as unstructured sparsity does not leverage hardware acceleration well.
- Weight pruning is an excellent strategy for model compression, especially for storage-constrained environments, but further optimizations (like structured pruning) are needed for significant speedups.

4.2 Quantization-Aware Training and Post-Training Quantization on Custom CNN

4.2.1 Objective

The experiment aims to evaluate the impact of Post-Training Quantization (PTQ) and Quantization-Aware Training (QAT) on the performance (accuracy, inference time, model size) of a custom CNN trained on the TensorFlow Flowers dataset. The goal is to convert the model into efficient formats for deployment on edge devices, using quantization techniques like INT8, FP16, and QAT (on all or partial layers).

4.2.2 Results

The detailed results are summarized in Tab. 4.3. In the following section, key observations and trends from these results are described and discussed. Additionally, a visualization comparing test accuracy, inference time, and model size is provided as Figure 4.1.

Accuracy

- Baseline TFLite using default PTQ achieves the highest accuracy (72.75%).
- PTQ models (INT8, FP16) have no or minimal accuracy drop.
- QAT-All model and its TFLite version show slightly lower test accuracy (70.84%), whereas QAT-ConvOnly and its TFLite version (65.94%) has higher accuracy drop and also high training accuracy (suggesting overfitting).

Inference Time

- Baseline is the slowest (0.0121 s/image).
- All PTQ models cut inference time almost by about 2×, with the best being Baseline TFLite (default PTQ).
- QAT models are slower during training, but their TFLite versions regain speed.
- The inference time for all quantized models (PTQ INT8, FP16, and QAT) is much lower, which represents a significant speed-up in inference. This is typical for quantized models, as lower precision (INT8 or FP16) allows faster computation on hardware optimized for these data types.

Model Size

- FP16 is slightly larger than INT8, which is expected as FP16 requires more bits to represent the weights and activations than INT8.
- Quantized models are 85–92% smaller than the baseline (from 23.36 MB down to 1.96 MB).

Raw QAT models (before TFLite) are larger because of training checkpoints and fake quantization nodes.

Tab. 4.3 QAT and PTQ of Custom CNN (Source: Author)

Model	Test Accuracy	Validation Accuracy	Train Accuracy	Model Size (MB)	Inference Time (s/image)
Baseline	72.21%	74.11%	68.19%	23.36	0.0121
Baseline TFLite (default PTQ)	72.75%	-	-	1.96	0.0070
PTQ FP16	72.21%	-	-	3.88	0.0076
PTQ INT8	72.21%	-	-	1.96	0.0079
QAT-All	70.84%	70.84%	75.37%	31.20	0.0268
QAT-All TFLite	70.84%	-	-	1.96	0.0076
QAT-ConvOnly	65.94%	65.67%	79.97%	24.89	0.0226
QAT-ConvOnly TFLite	65.94%	-	-	1.97	0.0091

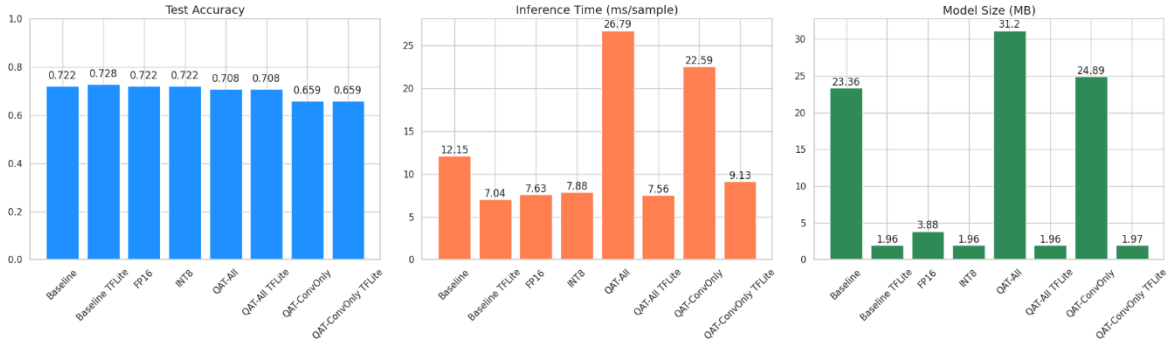

Figure 4.1 Comparison of QAT and PTQ on Custom CNN

4.2.3 Findings

- All quantized models (PTQ and QAT) have a much smaller file size compared to the baseline Keras model. This is a key advantage for deploying models on resource-constrained devices, such as mobile phones or embedded systems.
- Quantization significantly reduces model size and speeds up inference.
- PTQ (especially default/INT16) maintains comparable accuracy to the baseline with much better deployment efficiency.
- FP16 is ideal when precision matters and hardware supports FP16.
- QAT, while more training-intensive, provides a structured approach to quantization but doesn't outperform PTQ here.

4.3 Knowledge Distillation on Custom CNN Using Pretrained Model

4.3.1 Objective

The experiment aims to evaluate the effectiveness of Knowledge Distillation in improving the performance of a lightweight custom CNN (student model) by leveraging the soft-label predictions of a larger pretrained teacher model (InceptionV3). The objective is to determine whether transferring knowledge from a high-capacity model can enhance the student's generalization ability and accuracy, compared to training the student independently. The student model was trained using a combination of cross-entropy loss and soft targets from the teacher, modulated by two hyperparameters (Hinton et al. 2015):

- Temperature: Controls the softness of the teacher's predicted probabilities.
- Alpha: Balances the contribution of hard labels and soft targets in the loss function.

4.3.2 Results

The detailed results are summarized in Tab. 4.4. In the following section, key observations and trends from these results are described and discussed.

Accuracy

- The student model, which was overfitted (see Figure 4.2), achieved a test accuracy of 79.84%, lower than the teacher model's accuracy of 87.19%. To reduce number of parameters, the student model was built by modifying a custom CNN, replacing the Flatten layer with a GlobalAveragePooling2D layer. The teacher model, which had more parameters due to fine-tuning, was a pretrained InceptionV3 model.
- Knowledge Distillation improved the student's performance under certain settings:
    - Temperature 1.0, alpha 0.9 achieved a test accuracy of 83.92%, a 4% improvement over the student baseline and shows no sign of overfitting (Figure 4.3).
    - Temperature 5.0, alpha 0.5 achieved 81.74% test accuracy with a balanced train accuracy (80.69%).
- High alpha values (closer to 0.9) generally improved performance, confirming that giving higher weight to teacher signals is beneficial.
- Very low alpha (e.g., 3.0 temperature, 0.1 alpha) or extreme temperatures (e.g., 20.0 temperature) resulted in poor performance (test accuracy as low as 50.41% and 52.86%), suggesting that the student struggled to learn meaningful soft targets.

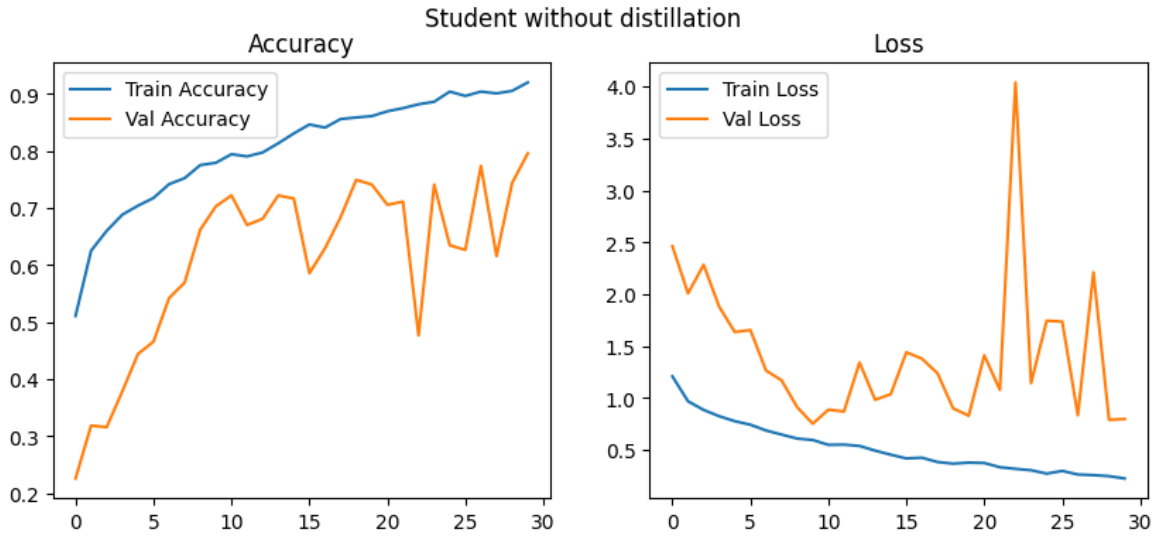

Figure 4.2 Student Model - Accuracy of Modified Custom CNN

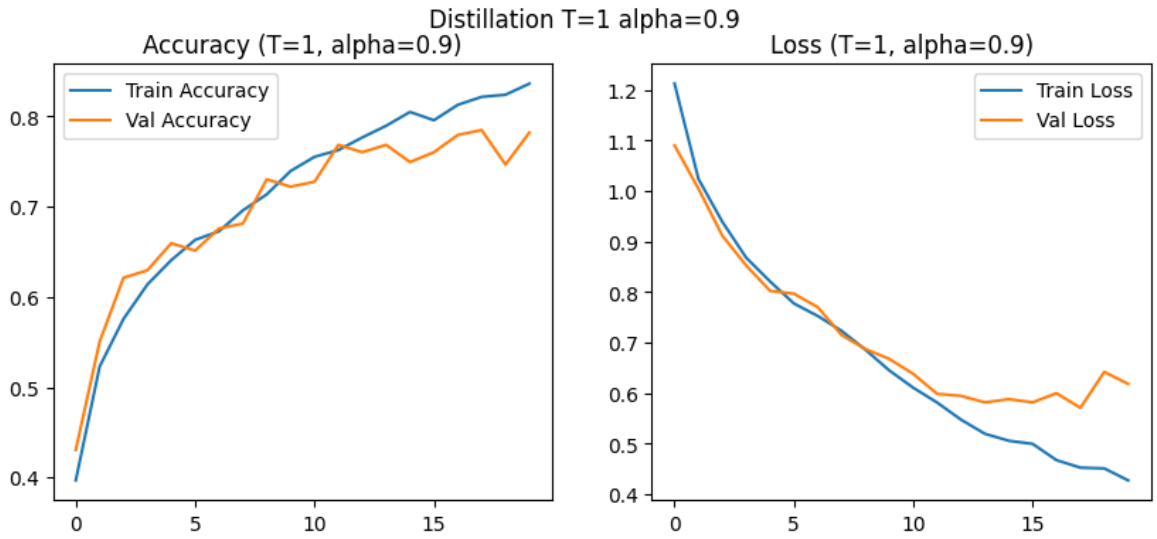

Figure 4.3 Distilled Model - Accuracy

Inference Time

- The distilled model maintained a lower inference time (0.08 s to 0.09 s) compared to the teacher (0.13 s).
- Among the distilled models, the fastest inference times were recorded around 0.0817 s (temperature 5.0, alpha 0.5) and 0.0809 s (temperature 3.0, alpha 0.7).
- This makes the distilled students highly suitable for real-time or resource-constrained deployments.

Model Size

- The teacher model (InceptionV3) had a significantly larger size (87.78 MB).
- The student and all distilled variants remained compact (1.84 MB), confirming that knowledge distillation increased the student's accuracy without increasing its model size, enabling significant efficiency gains compared to the large teacher model.

Tab. 4.4 Knowledge Distillation of Custom CNN (Source: Author)

Distillation Model (Temperature – Alpha)	Test Accuracy	Validation Accuracy	Train Accuracy	Inference Time (s/image)	Model Size
Teacher (InceptionV3)	87.19%	86.10%	92.64%	0.1286	87.777930
Student Baseline	79.84%	79.56%	91.99%	0.1023	1.836990
1.0 - 0.9	83.92%	78.47%	82.15%	0.0925	1.836998
3.0 - 0.1	50.41%	47.68%	41.52%	0.0837	1.837013
3.0 - 0.7	78.75%	76.57%	82.59%	0.0810	1.837013

3.0 - 0.9	81.2%	76.02%	82.63%	0.0840	1.837013
5.0 - 0.5	81.74%	78.47%	80.69%	0.0817	1.837013
5.0 - 0.7	78.75%	77.93%	83.14%	0.0827	1.837013
5.0 - 0.9	79.56%	79.29%	80.65%	0.0858	1.837013
7.0 - 0.3	80.38%	77.93%	81.64%	0.0904	1.837013
10.0 - 0.1	56.68%	53.95%	46.66%	0.0851	1.837021
10.0 - 0.4	79.84%	77.92%	80.10%	0.0848	1.837021
10.0 - 0.9	77.11%	77.38%	77.55%	0.0881	1.837021
20.0 - 0.1	52.86%	51.49%	48.16%	0.0913	1.837021

4.3.3 Findings

- Knowledge Distillation improved the performance of the compact student network, bringing it closer to the teacher's accuracy.
- Temperature 1.0 - Alpha 0.9 and Temperature 5.0 - Alpha 0.5 configurations performed best, achieving high test and validation accuracies while keeping inference time very low.
- Low alpha values or extremely high temperatures impaired learning from soft targets, leading to underperformance.
- Overall, distilled students demonstrated an excellent balance between accuracy, speed, and model size, making them very efficient for deployment scenarios.

4.4 Post-Training Quantization on Pretrained Model

4.4.1 Objective

This experiment evaluates the effects of post-training quantization (PTQ) techniques, (specifically INT8, FP16, and TFLite's default optimization) on an InceptionV3 model trained on the Stanford Dogs dataset. The goal is to reduce model size and improve inference time while maintaining classification accuracy suitable for deployment on edge devices.

4.4.2 Results

The detailed results are summarized in Tab. 4.5. In the following section, key observations and trends from these results are described and discussed. A visualization comparing test accuracy, inference time, and model size is provided (see Figure 4.4).

Accuracy

- All quantized models performed similarly to the FLOAT32 baseline in test accuracy, with slight improvements that may suggest potential regularization effects from quantization. However, this could also be attributed to the smaller test dataset used for evaluation.
- Both FP16 and Baseline TFLite (default PTQ) achieved the highest test accuracy (90.60%), closely followed by INT8 (90.50%).
- The full precision baseline model also performed well (90.17%) but with significantly higher resource requirements.

Inference Time

- Quantized models demonstrated faster inference compared to the full precision model, with up to a 1.5× improvement.
- Baseline TFLite (default PTQ) was the fastest (0.1874 s/image), slightly ahead of INT8 (0.1912 s/image) and FP16 (0.1949 s/image).
- The baseline model had the slowest inference time at 0.2910 s/image.

Model Size

- Post-training quantization considerably reduced model size:
    - From 90.58 MB (baseline) to 21.6–42.7 MB.
    - Baseline TFLite (default PTQ) and INT8 yielded the most compact models (21.6–21.8 MB).
    - FP16 remained larger (42.7 MB) due to its 16-bit representation.

Tab. 4.5 PTQ of Pretrained Model (Source: Author)

Model	Test Accuracy	Train Accuracy	Val Accuracy	Size (MB)	Inference Time (s/image)
Baseline	90.17%	89.46%	90.50%	90.59	0.2910
Baseline TFLite (default PTQ)	90.60%	-	-	21.60	0.1874
FP16	90.60%	-	-	42.66	0.1949

INT8	90.50%	-	-	21.81	0.1912

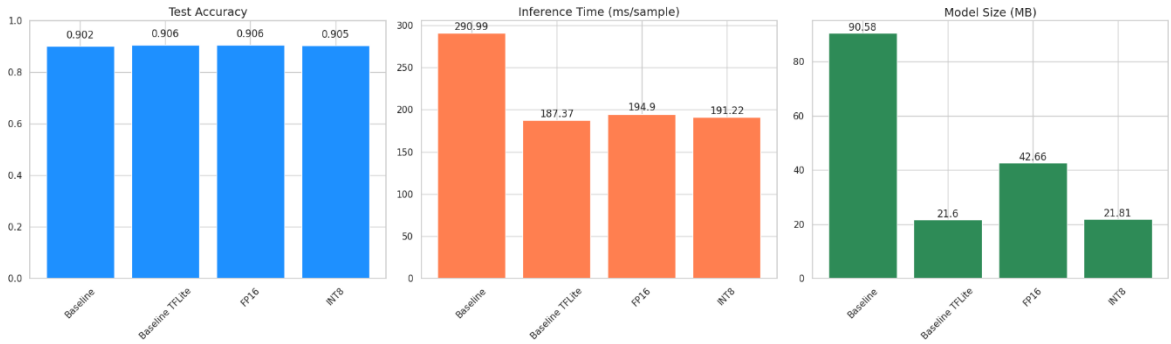

Figure 4.4 Comparison of PTQ on Pretrained Model

4.4.3 Findings

- Post-training quantization offers substantial benefits in size and speed with minimal (or no) impact on accuracy.
- FP16 and Baseline TFLite (default PTQ) delivered the highest accuracy, with only marginal differences in speed.
- INT8 stands out as a strong option for scenarios where minimizing model size and maintaining hardware-friendly inference is crucial.

4.5 Quantization-Aware Training on Pretrained Model

4.5.1 Objective

This experiment investigates the impact of Quantization Aware Training (QAT) on two convolutional neural network architectures - InceptionV3 and MobileNetV2 - trained on a classification task. The primary aim is to assess how QAT affects model accuracy, inference time, and storage size before and after conversion to TFLite format.

4.5.2 Results

The summarized performance metrics for all QAT-trained models, including their TFLite-converted versions, are presented in Tab. 4.6. The analysis considers training, validation, and test accuracy, model size, and per-image inference time. MobileNetV2[38] was included to provide a comparison and to address the unsatisfactory performance observed with InceptionV3.

[38] The corresponding baseline model is available in *BASELINE_STANFORD_DOGS*.ipynb in Appendix A.

Baseline models used for comparison have all layers frozen, as QAT-compatible baselines with all layers unfrozen were not evaluated in this experiment. QAT training and evaluation were conducted using an NVIDIA A100 GPU, while baseline experiments were performed on an NVIDIA T4 GPU.

Accuracy

- MobileNetV2 demonstrates superior generalization performance, achieving a test accuracy of 73.18% post-QAT, significantly higher than InceptionV3 (43.62%).
- QAT InceptionV3 exhibits high training accuracy (96.26%) but poor validation (41.25%) and test accuracy, suggesting severe overfitting.
- TFLite conversion retains test accuracy closely, with MobileNetV2 TFLite preserving accuracy (72.60%) and InceptionV3 TFLite showing a slight drop (41.80%).

Inference Time

- MobileNetV2 offers fast inference, with QAT and TFLite models achieving 0.1919 s and 0.0130 s per image respectively—ideal for real-time edge deployment.
- InceptionV3, despite being more complex, shows significantly slower inference (0.4806 s for QAT, 0.1619 s for TFLite).

Model Size

- TFLite conversion reduces model sizes drastically:
  - MobileNetV2 shrinks from 40.75 MB to 2.93 MB.
  - InceptionV3 reduces from 342.93 MB to 21.90 MB.
- These reductions demonstrate TFLite's effectiveness in compressing QAT models, especially beneficial for mobile and embedded systems.

Tab. 4.6 QAT on InceptionV3 and MobileNetV2 (Source: Author)

Model	Test Accuracy	Train Accuracy	Val Accuracy	Size (MB)	Inference Time (s/image)
InceptionV3 (GPU T4)	90.95%	89.29%	90.08%	85.89	0.0855
MobileNetV3 (GPU T4)	82.36%	78.32%	81.83%	10.33	0.0856
QAT MobileNetV2	73.18%	99.04%	72.58%	40.75	0.1919
QAT InceptionV3 TFLite	41.80%	-	-	21.90	0.1619
QAT MobileNetV2 TFLite	72.60%	-	-	2.93	0.0130

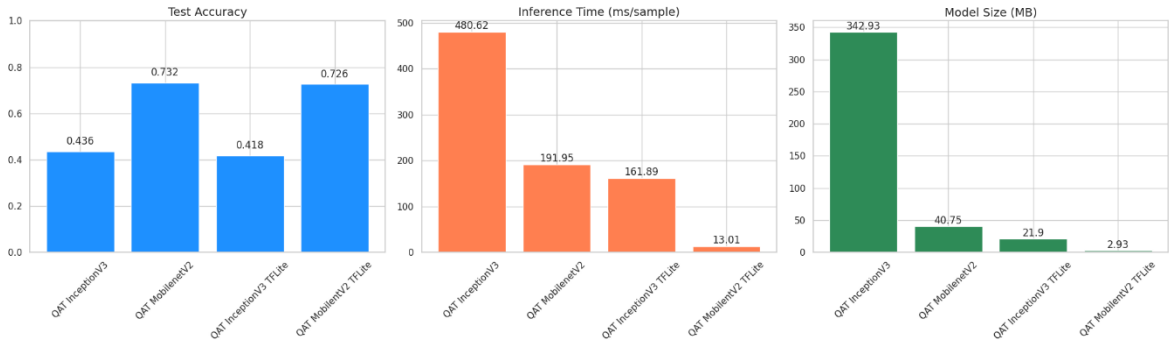

Figure 4.5 Comparison of QAT on Pretrained Model

4.5.3 Findings

- MobileNetV2 consistently outperforms InceptionV3 in generalization, size, and speed, making it the preferred model for QAT-based deployment.
- QAT introduces minimal degradation in test accuracy post-TFLite conversion, confirming its robustness for real-world applications.
- Inference time and storage size are significantly improved with TFLite—especially for MobileNetV2—without compromising accuracy, confirming its edge-readiness.
- InceptionV3, though accurate on training data, suffers from overfitting and high latency, limiting its suitability for lightweight scenarios.

4.5.4 Limitations and Challenges

While the experiment successfully demonstrated the benefits of Quantization Aware Training (QAT) for improving model efficiency, several limitations and technical challenges were encountered during implementation.

Hardware and Software Configuration

- The experiment was conducted on Google Colab using an NVIDIA A100 GPU, with TensorFlow version 2.14.1.
- This setup provided adequate GPU acceleration for training but also exposed compatibility and performance issues during model tuning.

Overfitting in Complex Models

- The InceptionV3 and MobileNetV2 models showed a severe overfitting problem, achieving high training accuracy, but poor validation and test accuracy.
- Attempts to mitigate overfitting (e.g., through dropout, data augmentation, fine-tuning or early stopping) were constrained due to limited experimentation time and slow retraining cycles.

TensorFlow Version Constraints

- Efforts to downgrade TensorFlow for compatibility with legacy quantization APIs led to dramatically longer training times, even with using A100 GPU.

- These performance regressions made hyperparameter optimization infeasible, as training a single model instance became prohibitively time-consuming.

4.6 Weight Pruning on Pretrained Model

4.6.1 Objective

This experiment aims to evaluate the effects of weight pruning on the performance of a more complex model, trained using a broader hyperparameter search space. The key objective is to determine how different sparsity schedules impact test accuracy, model compression, inference time, and generalization. Unstructured weight pruning was applied, and results were compared against a baseline with no pruning.

4.6.2 Results

The detailed results of hyperparameter searches for pruning are presented in Tab. 4.7. and Tab. 4.8.

Accuracy

- The unpruned model achieved the highest test accuracy of 89.84%, with strong validation and training performance.
- The best pruned model in terms of test accuracy (88.18%) used a 0.1–0.3 sparsity schedule, performing only 1.85% lower than the baseline.
- Aggressive pruning (e.g., 0.7–0.9) led to severe degradation in performance, dropping test accuracy to 1.38%, suggesting excessive loss of key weights.
- Mid-range sparsity schedules (0.1–0.5, 0.3–0.5, 0.4–0.5) yielded moderate test accuracies (between 65.50%–69.80%), though consistently underperformed relative to the baseline.

Model Size and Compression

- The original model size (86.15 MB) remained constant across configurations due to unchanging number of parameters, but gzip compression revealed true sparsity benefits:
    - From 79.24 MB (no pruning) down to 16.35 MB at 90% sparsity (only 2.28M non-zero parameters).
    - This highlights the potential of pruning to reduce storage without retraining smaller models from scratch.

Inference Time

- Due to unstructured pruning, extremely sparse models did not show speedups.

Tab. 4.7 Weight Pruning on Pretrained Model – Accuracy (Source: Author)

Sparsity (Init - Final)	Test Accuracy	Validation Accuracy	Train Accuracy	Accuracy Drop	Inference Time (s/image)
Baseline Model	89,84%	88,54%	90,56%	-	0.0881
0.1 - 0.3	88,18%	86,75%	88,64%	-1.85%	0.0910
0.1 - 0.5	69,74%	73,54%	68,94%	-22.35%	0.0907
0.3 - 0.5	69,79%	68,29%	69,65%	-22.26%	0.1056
0.4 - 0.5	68,57%	67,46%	68,72%	-23.70%	0.0910
0.5 - 0.5	65,55%	63,42%	63,80%	-27.02%	0.0918
0.2 - 0.6	28,73%	45,46%	31,99%	-68.03%	0.0904
0.3 - 0.7	2,35%	11,88%	2,23%	-87.38%	0.0914
0.5 - 0.7	2,48%	2,25%	2,38%	-87.23%	0.0915
0.7 - 0.9	1,38%	1,04%	0,81%	-88.65%	0.0918

Tab. 4.8 Weight Pruning on Pretrained Model – Size (Source: Author)

Sparsity (Init - Final)	Gzipped Model Size (MB)	Nonzero Parameters	Weight Percentage
Baseline Model	79.24	22 358 168	100%
0.1 - 0.3	62.02	15 666 331	70%
0.1 - 0.5	48.24	11 205 096	50%
0.2 - 0.6	40.92	8 974 475	40%
0.3 - 0.5	48.24	11 205 096	50%
0.3 - 0.7	33.25	6 743 861	30%
0.4 - 0.5	48.25	11 205 096	50%
0.5 - 0.5	48.25	11 205 096	50%

0.5 - 0.7	33.24	6 743 861	30%
0.7 - 0.9	16.35	2 282 645	10%

4.6.3 Findings

- Moderate pruning (final sparsity up to 30%) preserves most of the model's accuracy while achieving reasonable compression.
- High sparsity levels introduce drastic accuracy drops, likely due to excessive loss of key features or neurons.
- Compression gains (as seen in gzipped model sizes) are significant even with modest pruning.

4.6.4 Limitations and Challenges

Software Limitations

- TensorFlow 2.14.1 lacks efficient runtime support for sparse model inference on GPUs, meaning compressed models did not gain proportional speedups despite pruning.
- Attempting to downgrade TensorFlow to use certain model optimization tools caused severe slowdowns in training time, making the pruning workflow practically unmanageable.

Hyperparameter Optimization

- Due to the long training time and environment instability with downgraded TensorFlow versions, it was impossible to perform thorough hyperparameter tuning, including batch size adjustment, regularization strength, or optimizer selection.
- All pruning experiments were constrained low number of epochs, potentially leaving better-performing configurations unexplored.

4.7 Discussion

The experiments conducted across quantization, knowledge distillation, and pruning provide a comprehensive view of how various compression techniques can enhance the deploy ability of CNNs without heavily compromising performance. Each approach offered unique strengths and trade-offs, contributing valuable insights into optimizing models for edge environments.

Quantization techniques such as Post-Training Quantization (PTQ) and Quantization-Aware Training (QAT) are crucial for deploying deep learning models on edge devices, offering substantial improvements in model size and inference. In experiments with a custom CNN on the TensorFlow Flowers dataset, PTQ (INT8, FP16) reduced model size by over 90% and halved inference time without degrading accuracy, while QAT models performed similarly in size and speed but showed slight accuracy drops and risk of overfitting when applied selectively. Similar trends were observed with the pretrained InceptionV3 model on the Stanford Dogs dataset, where PTQ INT8 and FP16 models achieved test accuracies comparable to the FP32 baseline (90.50% and 90.60% vs. 90.17%), while offering up to 1.5× faster inference and reducing model size by over 75%.

In contrast, QAT on pretrained MobileNetV2 and InceptionV3 showed that MobileNetV2 is better suited for QAT-based deployment—achieving 73.18% test accuracy and compressing from 40.75 MB to just 2.93 MB with fast inference (0.0130 s/image), whereas InceptionV3 suffered from overfitting and slow performance (0.4806 s/image). A simple implementation from TensorFlow Model Optimization (n.d.-c) of MNIST example with Keras showed improved accuracy post-QAT (95.84% vs. 95.51%). Collectively, these results demonstrate that while both PTQ and QAT offer deployment advantages, their effectiveness depends on model architecture and quantization strategy.

Additionally, Knowledge Distillation was shown to enhance performance of lightweight student models. A compact custom CNN distilled from an InceptionV3 teacher improved from a baseline accuracy of 79.84% to 83.92% (with temperature 1.0 and alpha 0.9), without increasing model size (1.84M parameters) or compromising speed (0.08 s/image), offering a strong balance between efficiency and generalization. External studies reinforce these findings. Chen et al. (2018) achieved over 50% size reduction on CIFAR-100 while Khan et al. (2022) compressed a ResNet-50 teacher 15× into a fast, accurate DSNet for melanoma detection.

Weight pruning was applied to both a custom CNN trained on the TensorFlow Flowers dataset and a more complex pretrained model to evaluate its impact on accuracy, model size, inference time, and parameter sparsity. In the custom CNN, moderate pruning (up to 90% parameter reduction) not only maintained but slightly improved test accuracy (+1.36%), while significantly reducing gzipped model size from 7.19 MB to 1.30 MB. Inference time gains, however, were marginal due to the use of unstructured pruning. In contrast, the pretrained model exhibited more sensitivity to pruning. While light pruning (0.1–0.3 sparsity) caused only a minor drop in test accuracy (–1.85%), higher sparsity levels resulted in severe performance degradation (up to –88.65%), despite reducing model size from 79.24 MB to 16.35 MB. These outcomes were compared with the work of Han et al. (2015), who demonstrated that pruning LeNet-5 on MNIST reduced parameters from 431 000 to 36 000 (over 90%) without compromising accuracy. Overall, while pruning proves effective for model compression, especially in simpler architectures, achieving both high compression and preserved performance in complex models requires more sophisticated strategies and better hardware/software integration.

In summary, all explored methods succeeded in making models more efficient for deployment, each with specific advantages. Post-training quantization and knowledge distillation stood out as the most balanced approaches for real-world deployment, offering strong performance with minimal engineering overhead. Weight pruning showed promise, particularly in storage-constrained settings, though benefits in runtime efficiency remain limited without structured sparsity or hardware support.

Conclusions

This thesis has explored advanced techniques for enhancing the efficiency of deep learning models, specifically Quantization-Aware Training (QAT), Post-Training Quantization (PTQ), and Knowledge Distillation (KD), to improve the performance of convolutional neural networks (CNNs) for image classification tasks. Through a series of experiments conducted across various datasets, the goal was to answer three key research questions regarding the impact of these techniques on model accuracy, size, and inference time.

The experiments demonstrated that weight pruning has a variable effect depending on the architecture and the extent of pruning applied. In simpler, custom CNN architectures, moderate pruning (up to 90%) resulted in significant reductions in model size and a slight improvement in accuracy, highlighting pruning as an effective deep learning technique for model compression. However, when applied to more complex pretrained models, excessive pruning led to substantial performance degradation, particularly in terms of accuracy, underscoring the need for more sophisticated pruning strategies or hardware optimization. In terms of inference time, unstructured pruning had little to no effect. To achieve meaningful improvements, techniques like structured pruning or the use of specialized hardware would be necessary.

Quantization techniques were found to provide substantial benefits in terms of model size and inference time. Particularly using PTQ achieved up to 75%-90% reduction in model size and up to 1.5-8× faster inference while maintaining competitive accuracy. In contrast, QAT, while offering similar benefits in size and speed, showed more nuanced results, with certain configurations experiencing slight accuracy degradation or signs of overfitting, particularly when applied to specific layers.

Knowledge distillation proved successful in transferring knowledge from a larger teacher model to a smaller student model, leading to improvements in classification accuracy without increasing model size or compromising speed. In the case of a compact CNN distilled model from an InceptionV3 teacher, the accuracy increased from 79.84% to 83.92%, demonstrating the potential of this technique to improve performance in smaller, more efficient models. This finding aligns with previous research and suggests that knowledge distillation is an effective approach for creating efficient and accurate models.

Weight pruning is a promising approach for reducing model size, but its impact on accuracy and inference time varies across different architectures and pruning strategies. Quantization, particularly PTQ, offers a balanced trade-off between model size, speed, and accuracy, making it highly suitable for deployment on edge devices. Knowledge distillation, on the other hand, proved effective in enhancing the performance of smaller models, making it a valuable tool for optimizing lightweight models without compromising classification accuracy.

Despite these encouraging results, the study faced several limitations. All experiments were conducted exclusively in TensorFlow and run on Google Colab, which imposes restrictions in terms of GPU availability, RAM, session durations, and overall computational resources. These constraints limited the depth and scale of experimentation, particularly in terms of hyperparameter optimization and testing more complex architectures. The exclusive use of TensorFlow also narrows the generalizability of the results, as other frameworks like PyTorch may yield different outcomes. Additionally, while TF Flowers and Stanford Dogs are suitable for benchmarking, they are moderate in scale and complexity, and future work should explore how these optimization techniques perform on larger and more diverse datasets.

Building upon this research, future studies should consider conducting experiments in other deep learning frameworks to assess the consistency and portability of the observed performance gains. Furthermore, experiments could be replicated in more flexible and powerful environments, such as local high-performance GPU setups or cloud. Broader dataset selection—such as ImageNet, TinyImageNet, or domain-specific datasets like those used in medical or satellite imaging—would allow for testing under more complex and realistic conditions. It would also be valuable to assess how these optimization techniques perform across devices with varying memory and compute capabilities. Lastly, these findings suggest that combining the optimization techniques could lead to significant improvements in model efficiency for CNNs across various classification tasks.

List of references

Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., Levenberg, J., Mané, D., Monga, R., Moore, S., Murray, D., Olah, C., Schuster, M., Shlens, J., Steiner, B., Sutskever, I., Talwar, K., Tucker, P., Vanhoucke, V., Vasudevan, V., Viégas, F., Vinyals, O., Warden, P., Wattenberg, M., Wicke, M., Yu, Y., & Zheng, X. (2015). *TensorFlow: Large-scale machine learning on heterogeneous systems*. https://www.tensorflow.org/

Belcic, I., & Stryker, C. (2025, May 5). *What are model parameters?* IBM. https://www.ibm.com/think/topics/model-parameters

Bisong, E. (2019). *Google colaboratory*. In Building machine learning and deep learning models on google cloud platform: a comprehensive guide for beginners (pp. 59-64). Berkeley, CA: Apress.

Borup, K. (2020, September 1). *Knowledge distillation*. Keras. https://keras.io/examples/vision/knowledge_distillation/

Cao, D., & Aref, S. (2025). *Enhancing Ultra-Low-Bit Quantization of Large Language Models Through Saliency-Aware Partial Retraining* (No. arXiv:2504.13932). arXiv. https://doi.org/10.48550/arXiv.2504.13932

Chen, W.-C., Chang, C.-C., Lu, C.-Y., & Lee, C.-R. (2018). *Knowledge Distillation with Feature Maps for Image Classification* (No. arXiv:1812.00660). arXiv. https://doi.org/10.48550/arXiv.1812.00660

Chenna, D. (2024, April 15). *Quantization of convolutional neural networks: Quantization analysis*. Edge AI and Vision Alliance. https://www.edge-ai-vision.com/2024/04/quantization-of-convolutional-neural-networks-quantization-analysis/

Chollet, F., & others. (2015). *Keras* [Computer software]. https://keras.io

Clark, B. (2024, July 29). *What is Quantization?*. https://www.ibm.com/think/topics/quantization

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). *Imagenet: A large-scale hierarchical image database*. In 2009 IEEE conference on computer vision and pattern recognition (pp. 248–255).

Dominguez, D. (2025). *Understanding Inference-Time Compute*. *Medium*. https://dominguezdaniel.medium.com/understanding-inference-time-compute-c6c6ca2e17a8

Esteva, A., Kuprel, B., Novoa, R. A., Ko, J., Swetter, S. M., Blau, H. M., & Thrun, S. (2017). *Dermatologist-level classification of skin cancer with deep neural networks*. Nature, 542(7639), 115–118. https://doi.org/10.1038/nature21056

Gholami, A., Kim, S., Dong, Z., Yao, Z., Mahoney, M. W., & Keutzer, K. (2021). *A Survey of Quantization Methods for Efficient Neural Network Inference* (No. arXiv:2103.13630). arXiv. https://doi.org/10.48550/arXiv.2103.13630

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

Google AI Edge. (2024a). *Evaluate the models*. TensorFlow Lite. https://ai.google.dev/edge/litert/models/post_training_quant#evaluate_the_models

Google AI Edge. (2024b). *Post-training integer quantization*. TensorFlow Lite. https://ai.google.dev/edge/litert/models/post_training_integer_quant

Google AI Edge. (2024c). *Post-training float16 quantization*. TensorFlow Lite. https://ai.google.dev/edge/litert/models/post_training_float16_quant

Google AI Edge. (2024d). *Full integer quantization of weights and activations*. TensorFlow Lite. https://ai.google.dev/edge/litert/models/post_training_quantization#full_integer_quantization_of_weights_and_activations

Gou, J., Yu, B., Maybank, S. J., & Tao, D. (2021). Knowledge Distillation: A Survey. *International Journal of Computer Vision*, *129*(6), 1789–1819. https://doi.org/10.1007/s11263-021-01453-z

Guo, Q., Wu, X.-J., Kittler, J., & Feng, Z. (2021). *Differentiable Neural Architecture Learning for Efficient Neural Network Design* (No. arXiv:2103.02126). arXiv. https://doi.org/10.48550/arXiv.2103.02126

Han, S., Mao, H., & Dally, W. J. (2016). *Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding* (No. arXiv:1510.00149). arXiv. https://doi.org/10.48550/arXiv.1510.00149

Han, S., Pool, J., Tran, J., & Dally, W. J. (2015). *Learning both Weights and Connections for Efficient Neural Networks* (No. arXiv:1506.02626). arXiv. https://doi.org/10.48550/arXiv.1506.02626

He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep Residual Learning for Image Recognition*. 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 770–778. https://doi.org/10.1109/CVPR.2016.90

Hinton, G., Vinyals, O., & Dean, J. (2015). *Distilling the Knowledge in a Neural Network* (No. arXiv:1503.02531). arXiv. https://doi.org/10.48550/arXiv.1503.02531

Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). *MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications* (No. arXiv:1704.04861). arXiv. https://doi.org/10.48550/arXiv.1704.04861

Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K. Q. (2017). *Densely Connected Convolutional Networks*. 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), *2261–2269*. https://doi.org/10.1109/CVPR.2017.243

Hugging Face. (n.d.). *Quantization*. Retrieved 4 May 2025, from https://huggingface.co/docs/optimum/en/concept_guides/quantization

Ioffe, Sergey, and Christian Szegedy. (2015). *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. https://doi.org/10.48550/arXiv.1502.03167.

Khan, M. S., Alam, K. N., Dhruba, A. R., Zunair, H., & Mohammed, N. (2022). *Knowledge Distillation approach towards Melanoma Detection*. Computers in Biology and Medicine, 146, 105581. https://doi.org/10.1016/j.compbiomed.2022.105581

Khosla, A., Jayadevaprakash, N., Yao, B., & Fei-Fei, L. (2011, June). *Novel dataset for fine-grained image categorization*. First Workshop on Fine-Grained Visual Categorization, IEEE Conference on Computer Vision and Pattern Recognition, Colorado Springs, CO.

Kornblith, Simon, Jonathon Shlens, and Quoc V. Le. (2019). *Do Better ImageNet Models Transfer Better?* https://doi.org/10.48550/arXiv.1805.08974.

Lang, J., Guo, Z., & Huang, S. (2024). *A Comprehensive Study on Quantization Techniques for Large Language Models* (No. arXiv:2411.02530). arXiv. https://doi.org/10.48550/arXiv.2411.02530

LeCun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W., & Jackel, L. D. (1989). *Backpropagation Applied to Handwritten Zip Code Recognition*. Neural Computation, 1 (4), 541–551. https://doi.org/10.1162/neco.1989.1.4.541

Shapiro, L. G., & Stockman, G. C. (2001). *Computer Vision*. Prentice Hall.

LeCun, Y., Jackel, L., Bottou, L., Brunot, A., Cortes, C., Denker, J., Drucker, H., Guyon, I., Muller, U., Sackinger, E., Simard, P., Vapnik, V., & Laboratories, B. (1995). *COMPARISON OF LEARNING ALGORITHMS FOR HANDWRITTEN DIGIT RECOGNITION*. International Conference on Artificial Neural Networks.

Liu, Z., Sun, M., Zhou, T., Huang, G., & Darrell, T. (2019). *Rethinking the Value of Network Pruning* (No. arXiv:1810.05270). arXiv. https://doi.org/10.48550/arXiv.1810.05270

Menghani, G. (2021). *Efficient Deep Learning: A Survey on Making Deep Learning Models Smaller, Faster, and Better*. ACM Computing Surveys, 55(12), 1–37. https://doi.org/10.1145/3578938

Nair, V., & Hinton, G. E. (2010). *Rectified Linear Units Improve Restricted Boltzmann Machines*. In Proceedings of the 27th international conference on machine learning (ICML-10) (pp. 807-814).

Neo, M. (2024). *A comprehensive guide to neural network model pruning*. Datature Blog. https://www.datature.io/blog/a-comprehensive-guide-to-neural-network-model-pruning

Or, A., Zhang, J., Smothers, E., Khandelwal, K., & Rao, S. (2024, July 30). *Quantization-aware training for large language models with PyTorch*. PyTorch Blog. https://pytorch.org/blog/quantization-aware-training/

Praveen, V. (2029). *Improving INT8 Accuracy Using Quantization Aware Training and the NVIDIA TAO Toolkit*. NVIDIA Technical Blog. https://developer.nvidia.com/blog/improving-int8-accuracy-using-quantization-aware-training-and-tao-toolkit/

PyTorch ExecuTorch. (n.d.). PyTorch. https://docs.pytorch.org/executorch-overview

PyTorch Mobile – PyTorch. (n.d.). https://pytorch.org/mobile/

PyTorch. (2019, October 9). *Quantization* (Last updated 2025, April 1). PyTorch Tutorials. Retrieved from https://pytorch.org/docs/main/quantization.html

Ray, J. (2024, December 25). *Quantization Aware Training (QAT) vs. Post-Training Quantization (PTQ)*. Better ML. https://medium.com/better-ml/quantization-aware-training-qat-vs-post-training-quantization-ptq-cd3244f43d9a

Russakovsky, O., et al. (2015). *ImageNet Large-Scale Visual Recognition Challenge*. International Journal of Computer Vision, 115(3), 211-252.

Simonyan, K., & Zisserman, A. (2015). *Very Deep Convolutional Networks for Large-Scale Image Recognition*. 3rd International Conference on Learning Representations (ICLR 2015), 1–14.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). *Dropout: a simple way to prevent neural networks from overfitting*. The journal of machine learning research, 15(1), 1929-1958.

Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., & Wojna, Z. (2015). *Rethinking the Inception Architecture for Computer Vision* (No. arXiv:1512.00567). arXiv. https://doi.org/10.48550/arXiv.1512.00567

Tan, M., & Le, Q. V. (2020). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks* (No. arXiv:1905.11946). arXiv. https://doi.org/10.48550/arXiv.1905.11946

Tan, M., & Le, Q. V. (2020). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks* (No. arXiv:1905.11946). arXiv. https://doi.org/10.48550/arXiv.1905.11946

TensorFlow Model Optimization Team. (2019, June 11*). TensorFlow model optimization toolkit - Post-training integer quantization. TensorFlow Blog*. https://blog.tensorflow.org/2019/06/tensorflow-integer-quantization.html

TensorFlow Model Optimization. (n.d.-a). *Pruning in Keras example*. TensorFlow. https://www.tensorflow.org/model_optimization/guide/pruning/pruning_with_keras

TensorFlow Model Optimization. (n.d.-b). *Quantization aware training*. TensorFlow. https://www.tensorflow.org/model_optimization/guide/quantization/training

TensorFlow Model Optimization. (n.d.-c). *Quantization aware training in Keras example*. TensorFlow. https://www.tensorflow.org/model_optimization/guide/quantization/training_example

The TensorFlow Team. (2019, January). *Flowers*. http://download.tensorflow.org/example_images/flower_photos.tgz

Veeling, B. S., Linmans, J., Winkens, J., Cohen, T., & Welling, M. (2018, September). *Rotation equivariant CNNs for digital pathology*. https://doi.org/10.1007/978-3-030-00934-2_24

White, C., Safari, M., Sukthanker, R., Ru, B., Elsken, T., Zela, A., Dey, D., & Hutter, F. (2023). *Neural Architecture Search: Insights from 1000 Papers* (No. arXiv:2301.08727). arXiv. https://doi.org/10.48550/arXiv.2301.08727

Yu, T., & Zhu, H. (2020). *Hyper-Parameter Optimization: A Review of Algorithms and Applications* (No. arXiv:2003.05689). arXiv. https://doi.org/10.48550/arXiv.2003.05689

Zhou, B., Lapedriza, A., Khosla, A., Oliva, A., & Torralba, A. (2017). *Places: A 10 million image database for scene recognition*. IEEE Transactions on Pattern Analysis and Machine Intelligence. IEEE.

Appendices

Appendix A: Source Code

File *BASELINE_TF_FLOWERS.ipynb* contains the baseline model implementation for the TF Flowers dataset.

File *BASELINE_STANFORD_DOGS.ipynb* provides the baseline model and other pretrained models, which were initially considered for the Stanford Dogs dataset.

File *TF_FLOWERS_pruning.ipynb* presents weight pruning with various parameters applied to the TF Flowers dataset using a custom CNN architecture.

File *TF_FLOWERS_quantization.ipynb* covers both Quantization Aware Training and Post Training Quantization on the TF Flowers dataset with a custom CNN architecture.

File *TF_FLOWERS_distillation.ipynb* illustrates Knowledge Distillation on the TF Flowers dataset using a combination of a custom CNN and pretrained InceptionV3 model.

File *STANFORD_DOGS_quantization_ptq.ipynb* implements Post Training Quantization on the Stanford Dogs dataset using a custom pretrained InceptionV3 model.

File *STANFORD_DOGS_quantization_qat.ipynb* shows Quantization Aware Training on the Stanford Dogs dataset using pretrained InceptionV3 and MobileNetV2 models.

File *STANFORD_DOGS_pruning.ipynb* demonstrates weight pruning on the Stanford Dogs dataset using pretrained models.

The files are available as attachments in the InSIS system.